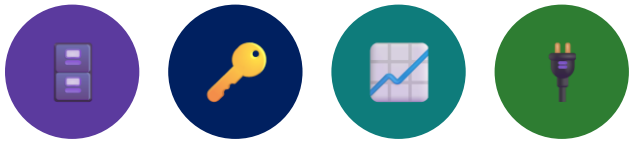
*"Model the data. Enforce the rules. Connect it all together."*

MODULE 37

# PostgreSQL Database Fundamentals

Design normalized PostgreSQL schemas from first principles -- tables, types, keys, constraints, indexes, and real connectivity from psql to Java.



TEKsystems Global Services

# WORKFORCE DEVELOPMENT

# MODULE 37 OVERVIEW

*Design robust, normalized PostgreSQL databases and connect them to real applications.*

Northstar CRM needs a real relational schema behind every screen -- customers, accounts, and addresses, modeled and constrained in PostgreSQL from day one.

## DURATION & LAB

### 1.5 Hours Theory

Relational fundamentals, data types, keys, constraints, relationships, indexes, sequences, and connectivity.

### 2 Hours Lab

Design and build a real PostgreSQL schema for the CRM customer and account platform.

### Scenario

Northstar CRM: CUSTOMER, ACCOUNT, ADDRESS, and status history, modeled and constrained end to end.

## MODULE ARC

### Understand the Model

Relational concepts, PostgreSQL architecture, and how data is organized into databases and schemas.

### Master Keys & Constraints

Primary keys, foreign keys, uniqueness, NOT NULL, CHECK, and normalized relationships.

### Connect and Govern

Indexes, sequences, psql, JDBC, users, roles, transactions, and enterprise design practice.

KEY TAKEAWAY  Total time: 3.5 hours (1.5 hours theory + 2 hours hands-on lab). By the end, you will design, constrain, and connect to a normalized PostgreSQL schema.

# WHY RELATIONAL DATABASES MATTER

*Learn how to design robust, normalized databases in PostgreSQL for real enterprise systems.*

### Design Better Databases

Model entities, attributes, and relationships before writing a single CREATE TABLE.

### Ensure Data Integrity

Enforce rules and consistency so the database itself rejects invalid data.

### Support Scalable Apps

Well-structured schemas hold up as data and traffic grow.

### Make Smarter Decisions

Clean, trustworthy data powers better reporting and analytics.

KEY TAKEAWAY  A well-designed PostgreSQL schema is the single foundation every reliable, secure, high-performance application is built on.

# Why Relational Databases Matter

Relational databases are the backbone of modern applications and enterprise systems.

### Data Integrity
Enforce rules, constraints, and relationships to keep data accurate and consistent.

### ACID Transactions
Ensure reliable operations with Atomicity, Consistency, Isolation, and Durability.

### Complex Relationships
Model real-world connections between data using foreign keys and relationships.

### Powerful Querying
Use SQL to query, analyze, and report data efficiently.

### Scalability
Handle growing data volumes and users with proven architectures and indexing.

### Security
Role-based access, permissions, and auditing protect sensitive data.


**Relational databases provide the structure, reliability, and performance needed to build secure, data-driven applications at scale.**

# MODULE LEARNING OBJECTIVES

*By the end of this module, you will be able to design, constrain, and connect to a normalized PostgreSQL database.*

1. Understand Fundamentals -- explain relational concepts and PostgreSQL's architecture.
2. Choose Data Types -- select the right PostgreSQL type for every column.
3. Create Structures -- create databases, schemas, and tables with confidence.
4. Enforce Integrity -- apply primary keys, foreign keys, and constraints correctly.
5. Model Relationships -- design one-to-one, one-to-many, and many-to-many links.
6. Normalize Data -- reduce redundancy while keeping queries practical.
7. Index for Performance -- apply B-Tree, unique, and composite indexes.
8. Connect Real Clients -- use psql, pgAdmin, and JDBC from a Java application.

KEY TAKEAWAY  Goal: design a normalized, constrained, and correctly indexed PostgreSQL schema that a real Java application can connect to safely.

MODULE 37: PostgreSQL Database Fundamentals

# Learning Objectives

By the end of this module, you will be able to:

**Understand PostgreSQL**
Explain PostgreSQL architecture and its key components.

**Design Database Structures**
Create databases, schemas, and tables using appropriate data types and constraints.

**Model Relationships**
Implement one-to-one, one-to-many, and many-to-many relationships using foreign keys.

**Apply Constraints and Keys**
Use primary keys, foreign keys, unique, not null, check constraints, and default values to ensure data integrity.

**Use Indexes and Sequences**
Create and manage indexes, sequences, and identity columns for performance and data consistency.

**Work with Users and Permissions**
Manage database users, roles, and permissions using the principle of least privilege.

**Apply Transactions and ACID Principles**
Demonstrate how transactions and ACID properties ensure reliable and consistent data operations.


**These fundamentals form the foundation for building robust, secure, and high-performance applications with PostgreSQL.**

# POSTGRESQL OVERVIEW

*PostgreSQL is a powerful, open-source, standards-compliant object-relational database system.*

### Open Source

Free, transparent, community- and enterprise-driven development with no licensing cost.

### Standards Compliant

Strong adherence to the SQL standard, plus rich PostgreSQL-specific extensions.

### Extensible

Custom types, functions, operators, and extensions (e.g., PostGIS) plug directly into the engine.

### ACID & MVCC

Full transactional integrity with Multi-Version Concurrency Control for high concurrency.

KEY TAKEAWAY  PostgreSQL combines standards-compliant SQL, strong ACID guarantees, and rich extensibility -- the engine this course, and its labs, are built on.

# PostgreSQL Overview

PostgreSQL
The world's most advanced open source database.

PostgreSQL is a **powerful, open source object-relational** database system.
It is known for **reliability, data integrity, extensibility,** and **performance.**

## Key Features

**ACID Compliant**
Ensures reliable transactions with full ACID support.

**High Performance**
Powerful query planner, indexes, and concurrency control.

**Advanced Data Types**
Supports JSON/JSONB, arrays, UUID, full-text search, and more.

**Reliability & Integrity**
Strong data integrity, foreign keys, constraints, and crash recovery.

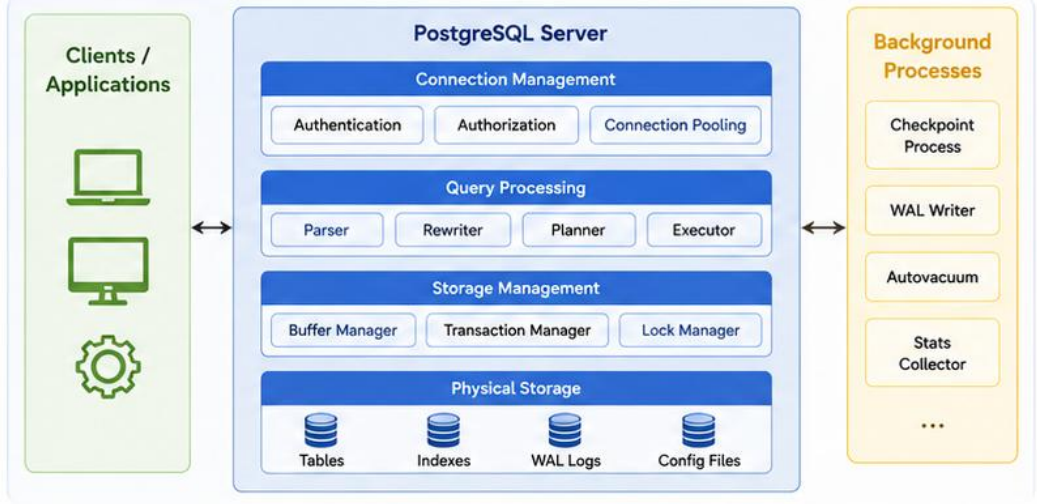
**Extensible**
Custom data types, functions, operators, and extensions.

**Open Source & Community**
Large global community and continuous improvement.

## Use Cases

- Enterprise Applications
- Analytics & Reporting
- Web & Mobile Applications
- AI/ML and Data Science
- JSON/Document Workloads

## PostgreSQL Architecture




## Why Organizations Choose PostgreSQL

- Proven Reliability
- Enterprise Ready
- Scalability
- Extensibility
- Cost Effective

PostgreSQL combines the **robustness** of enterprise databases with the **flexibility** of open source.

# POSTGRESQL ARCHITECTURE

*PostgreSQL uses a process-per-connection model with a shared storage engine.*

### 1 Clients / Apps

Connect to PostgreSQL over the network using a driver such as JDBC.

### 2 Postmaster

The supervisor process authenticates each connection and spawns a backend.

### 3 Backend Process

One dedicated OS process per connection handles that session's queries.

### 4 Shared Memory & Storage

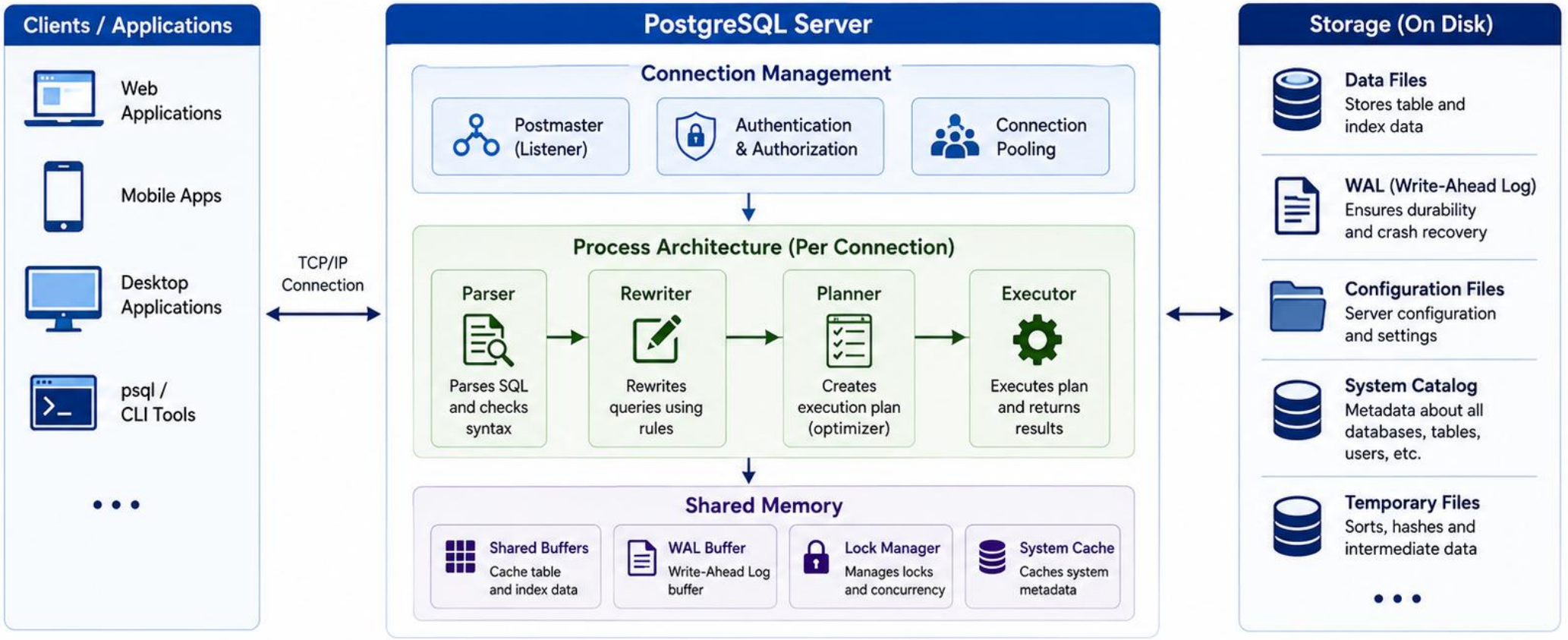
Shared buffers, the WAL, and a lock table are shared across all backend processes.

KEY TAKEAWAY  The Postmaster spawns one dedicated backend process per client connection; all processes share buffers, the write-ahead log, and a lock table.

# PostgreSQL Architecture

PostgreSQL

PostgreSQL uses a **client–server** architecture with a multi-process model for performance, reliability, and scalability.




**Key Takeaways**

- PostgreSQL uses a multi-process architecture for stability and performance.
- Each client connection has its own backend process.
- Shared memory and background processes work together to handle concurrent workloads efficiently.

# CLIENT-SERVER DATABASE ARCHITECTURE

*PostgreSQL always runs as a server process that clients connect to over the network -- never embedded in the application.*

| Role | Responsibility |
|---|---|
| **Server (postgres)** | Runs continuously, owns the data files, enforces every constraint and permission. |
| **Client** | psql, pgAdmin, DBeaver, or a Java app -- sends SQL, receives result sets. |
| **Protocol** | The PostgreSQL wire protocol carries queries and results over TCP, typically port 5432. |
| **Network Boundary** | Client and server can run on the same machine or across a network -- the protocol is identical either way. |

KEY TAKEAWAY  Every client -- human or application -- talks to the same PostgreSQL server process over the same wire protocol; the server is the single source of truth.

# Client–Server Database Architecture

PostgreSQL

PostgreSQL uses a **client–server model** where client applications request data and the PostgreSQL server processes the request, manages the data, and returns the results.

## CLIENTS

- Web Applications
- Mobile Applications
- Desktop Applications
- psql / CLI Tools
- • • •

## NETWORK (TCP/IP)

- Secure Connection
- Request / Response
- Standard Protocols

## POSTGRESQL SERVER

### Request Flow

1. **Receive Request**
   (Client sends SQL or API request)
2. **Process Request**
   Parser, planner, and optimizer generate execution plan
3. **Execute Query**
   Executor runs the plan and accesses data
4. **Return Results**
   Results are formatted and sent back to client

### Server Components

- **Connection Manager**
  Manages client connections, authentication, and concurrency
- **Query Processor**
  Parses, analyzes, optimizes, and prepares execution plans
- **Storage Manager**
  Manages disk I/O, buffer cache, and data files
- **Transaction Manager**
  Ensures ACID properties, concurrency control, and recovery

## DATABASE (ON DISK)

- **Data Files**
  Tables, indexes, and data
- **WAL (Write-Ahead Log)**
  Ensures durability and crash recovery
- **Configuration Files**
  Server configuration and settings
- **System Catalog**
  Metadata about tables, columns, users, and permissions
- **Temporary Files**
  Sorts, hashes, and intermediate data

## KEY TAKEAWAYS

- Clients communicate with the PostgreSQL server over the network using standard protocols.
- The server handles query parsing, optimization, execution, and transaction management.
- Data is stored on disk with strong durability, consistency, and reliability.
- This architecture enables scalability, security, centralized management, and data integrity.

# DATABASE VS SCHEMA

*A PostgreSQL server can host many databases; each database can hold many schemas, each a namespace for tables.*

| Level | What It Is | Typical Use |
|---|---|---|
| **Server / Cluster** | One running PostgreSQL instance. | One installation, many databases. |
| **Database** | An isolated collection of schemas, tables, and data. | One per application, e.g. crmdb. |
| **Schema** | A named namespace of tables inside one database. | Group by module or tenant, e.g. public, billing. |
| **Table** | The actual rows and columns of data. | customer, account, address. |

KEY TAKEAWAY  A database isolates data completely; a schema inside it is just an organizational namespace -- most PostgreSQL apps use one database and the default public schema.

# Database vs Schema

PostgreSQL

In PostgreSQL, a **database** is a container for schemas.
A **schema** is a namespace that contains database objects such as tables, views, and functions.

## PostgreSQL Server

**Server Instance**

- A PostgreSQL server can host multiple databases.

**Example Databases**

- employee_db
- sales_db
- hr_db
- analytics_db
- ...

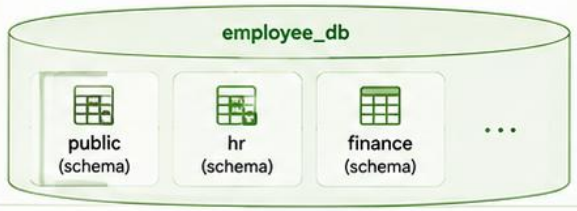
## DATABASE

A **database** is a top-level container in PostgreSQL.

- Has a unique name within the server instance
- Contains one or more schemas
- Represents a logical collection of data
- Has its own settings, users, and permissions

**Example**

```
-- Create a new database
CREATE DATABASE employee_db;

-- Connect to the database
\c employee_db;
```




Think of a **database** as a folder that can contain multiple schemas.

## SCHEMA

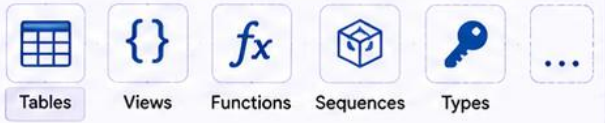
A **schema** is a namespace within a database.

- Organizes and groups database objects
- The default schema is `public`
- Multiple schemas help separate data by modules, teams, or business domains
- Supports the same object types as the public schema

**Example**

```
-- Create a new schema
CREATE SCHEMA hr;

-- Use the schema
SET search_path TO hr, public;
```

**Objects inside a schema (e.g., hr)**




Think of a **schema** as a drawer inside the database that holds related objects.

## Example View

Database: employee_db

**Schemas**

- public
  - employees
  - departments
  - locations
  - ... ...
- hr
  - employees
  - job_history
  - leave_balance
  - ... ...
- finance
  - accounts
  - transactions
  - invoices
  - ... ...

## KEY TAKEAWAYS

- A **database** is a container that can hold multiple schemas.
- A **schema** is a namespace that organizes database objects.
- Use multiple schemas to logically separate data by team or domain.
- Both databases and schemas have their own permissions and security.

# TABLES AND ROWS

*Data in PostgreSQL is organized into tables; each table is made of rows, and each row is one record.*

| customer_id | full_name | status | created_at |
|---|---|---|---|
| **1** | Amina Khan | ACTIVE | 2026-01-10 |
| **2** | Ravi Singh | PROSPECT | 2026-01-11 |

Table -- stores data about one entity or business object, e.g. customer.
Row (tuple) -- a single record or instance of that entity, e.g. one customer.
Column (attribute) -- a specific property of the entity, e.g. full_name or status.
Every row in a table shares the exact same set of columns.

KEY TAKEAWAY  A table stores one entity; each row is one instance of it; every row shares the exact same columns -- the foundation for everything that follows this module.

# Tables and Rows

PostgreSQL

In PostgreSQL, data is stored in **tables**. A table is made up of **rows** and **columns**.
Each row represents a record, and each column represents a **field** (attribute).

## KEY CONCEPTS

**Table**
A collection of related data organized in columns and rows.

**Row (Record)**
A single entry in a table. Each row has a unique set of values.

**Column (Field)**
A named attribute of the table. All values in a column are of the same data type.

**Analogy**
Think of a table as a spreadsheet: columns are the headers, rows are the data entries.

## EXAMPLE: employees TABLE

PRIMARY KEY

COLUMNS (FIELDS)

ROWS (RECORDS)

| employee_id (integer) | first_name (text) | last_name (text) | email (text) | department_id (integer) | hire_date (date) | salary (numeric) |
|---|---|---|---|---|---|---|
| 1 | John | Doe | john.doe@acme.com | 10 | 2023-01-15 | 75000.00 |
| 2 | Jane | Smith | jane.smith@acme.com | 20 | 2023-02-20 | 68000.00 |
| 3 | Michael | Brown | michael.brown@acme.com | 10 | 2023-03-05 | 72000.00 |
| 4 | Emily | Davis | emily.davis@acme.com | 30 | 2023-03-18 | 65000.00 |
| 5 | Daniel | Wilson | daniel.wilson@acme.com | 20 | 2023-04-01 | 70000.00 |

**SQL EXAMPLE**

```
CREATE TABLE employees (
    employee_id   SERIAL PRIMARY KEY,
    first_name      TEXT NOT NULL,
    last_name       TEXT NOT NULL,
    email           TEXT UNIQUE,
    department_id   INTEGER REFERENCES departments(department_id),
    hire_date       DATE NOT NULL,
    salary          NUMERIC(12,2)
);
```

```
-- Each column has a name and data type
-- Each row stores values for these columns
```

## WHY IT MATTERS

**Structured Data**
Organizes data in a structured and efficient way.

**Easy Retrieval**
Makes querying and filtering data simple and powerful.

**Data Integrity**
Constraints (like primary key, foreign key) help maintain accuracy.

**Scalability**
Designed to handle large volumes of data reliably.

## KEY TAKEAWAYS

- A table stores data in a structured format of rows and columns.
- Each row is a record. Each column is a field with a specific data type.
- Primary keys uniquely identify each row in the table.
- This structure enables efficient data management and powerful querying in PostgreSQL.

# COLUMNS AND DATA TYPES

*Every column has a declared data type that constrains exactly what values it can hold.*

### Type Enforcement

PostgreSQL rejects a value that does not match its column's declared type at insert time.

### Storage & Precision

The type determines how much space a value uses and how precisely it is stored.

### Operator Behavior

A column's type decides which operators and functions are valid against it, e.g. date arithmetic.

### Constraints Ride Along

NOT NULL, DEFAULT, and CHECK are attached per column alongside its type.

KEY TAKEAWAY  The declared type of a column is a contract: PostgreSQL enforces it on every insert and update, and it drives storage, precision, and which operations are valid.

# Columns and Data Types

PostgreSQL

In PostgreSQL, a table is made up of **columns** (fields).
Each column has a name and a **data type** that defines the kind of values it can store.

## EXAMPLE TABLE: employees

COLUMNS (FIELDS)

| employee_id (INTEGER) | first_name (TEXT) | last_name (TEXT) | email (TEXT) | department_id (INTEGER) | hire_date (DATE) | salary (NUMERIC) |
|---|---|---|---|---|---|---|
| 1 | John | Doe | john.doe@acme.com | 10 | 2023-01-15 | 75000.00 |
| 2 | Jane | Smith | jane.smith@acme.com | 20 | 2023-02-20 | 68000.00 |
| 3 | Michael | Brown | michael.brown@acme.com | 10 | 2023-03-05 | 72000.00 |
| 4 | Emily | Davis | emily.davis@acme.com | 30 | 2023-03-18 | 65000.00 |
| 5 | Daniel | Wilson | daniel.wilson@acme.com | 20 | 2023-04-01 | 70000.00 |

ROWS (RECORDS)

Think of a table as a spreadsheet:
Columns are the headers, rows are the data entries.

### KEY POINTS

- Each column has a name and a data type.
- The data type determines what kind of values the column can store.
- Choosing the right data type improves data integrity and performance.

## COMMON POSTGRESQL DATA TYPES

| CATEGORY | DATA TYPE | DESCRIPTION | EXAMPLE |
|---|---|---|---|
| Numeric | INTEGER | Whole numbers (4 bytes) | 42 |
| | BIGINT | Large whole numbers (8 bytes) | 9223372036854775807 |
| | NUMERIC(p,s) | Exact numeric with precision and scale | 12345.67 |
| | REAL / DOUBLE PRECISION | Floating-point numbers | 3.14159 |
| Character | TEXT | Variable-length character string | 'Hello PostgreSQL' |
| | VARCHAR(n) | Variable-length with limit | 'Alice' |
| | CHAR(n) | Fixed-length character string | 'USA ' |
| Boolean | BOOLEAN | Stores true or false | true / false |
| Date / Time | DATE | Calendar date (YYYY-MM-DD) | 2024-05-20 |
| | TIME | Time of day (HH:MM:SS) | 14:30:00 |
| | TIMESTAMP | Date and time without time zone | 2024-05-20 14:30:00 |
| | TIMESTAMPTZ | Date and time with time zone | 2024-05-20 14:30:00+00 |
| Other | UUID | Universally unique identifier | 550e8400-e29b-41d4-a-716-446655440000 |
| | JSON / JSONB | JSON data (JSONB is binary JSON) | { "name": "Alice" } |
| | BYTEA | Binary data | \xDEADBEEF |

## KEY TAKEAWAYS

- Columns define the structure of a table.
- Data types ensure only valid values are stored.
- Strong data types improve accuracy and reliability.
- Proper data types lead to better performance.

# POSTGRESQL DATA TYPES OVERVIEW

*PostgreSQL ships a rich built-in type system -- choosing the right one is a real design decision.*

| Category | Examples | Typical Use |
|---|---|---|
| **Numeric** | INTEGER, BIGINT, NUMERIC, REAL | Counts, IDs, exact money amounts. |
| **Character** | VARCHAR(n), TEXT, CHAR(n) | Names, emails, free text, fixed codes. |
| **Boolean** | BOOLEAN | True/false flags, e.g. is_active. |
| **Date/Time** | DATE, TIME, TIMESTAMPTZ, INTERVAL | Timestamps, durations, schedules. |
| **Identifier** | UUID | Globally unique, non-sequential public identifiers. |
| **Semi-Structured** | JSON, JSONB | Flexible, schema-light document data. |

KEY TAKEAWAY  Six broad type categories cover almost every column PostgreSQL schemas need -- the next six slides walk each one with real examples.

# PostgreSQL Data Types Overview

PostgreSQL provides a rich set of native data types to store different kinds of data efficiently and safely.

## 123 Numeric Types

For numbers, integers, decimals, and arbitrary precision values.

| Type | Example |
|---|---|
| SMALLINT | 32767 |
| INTEGER | 2147483647 |
| BIGINT | 9223372036854775807 |
| NUMERIC(p,s) | 12345.6789 |
| DECIMAL(p,s) | 99.99 |
| REAL | 3.14159 |
| DOUBLE PRECISION | 2.718281828459045 |

## A Character Types

For storing text, strings, and fixed-length character data.

| Type | Example |
|---|---|
| CHAR(n) | 'ABC' |
| VARCHAR(n) | 'Hello World' |
| TEXT | 'This is a text value of any length' |
| CITEXT | 'Case Insensitive Text' |

## Date and Time Types

For storing date, time, timestamp, and interval data.

| Type | Example |
|---|---|
| DATE | 2025-05-18 |
| TIME | 14:30:00 |
| TIME WITH TIME ZONE | 14:30:00+05:30 |
| TIMESTAMP | 2025-05-18 14:30:00 |
| TIMESTAMP WITH TIME ZONE | 2025-05-18 14:30:00+05:30 |
| INTERVAL | '2 days 3 hours' |

## Boolean Type

For storing true/false values.

| Type | Example |
|---|---|
| BOOLEAN | true / false |

## { } UUID Type

For storing universally unique identifiers.

| Type | Example |
|---|---|
| UUID | 550e8400-e29b-41d4-a716-446655440000 |

## JSON JSON Types

For storing semi-structured and flexible JSON data.

| Type | Example |
|---|---|
| JSON | '{"name": "John", "age": 30}' |
| JSONB | '{"role": "admin", "active": true}' |

## Key Takeaways

- Choose the right data type to ensure data integrity and performance.
- Use NUMERIC/DECIMAL for exact financial or precise calculations.
- Use TEXT for large text data.
- Use TIMESTAMP WITH TIME ZONE for timezone-aware applications.
- Use UUID for globally unique identifiers.
- Use JSONB for efficient querying and indexing of JSON data.

### Pro Tip

Use the most appropriate data type from the beginning—changing data types later can be expensive!

PostgreSQL is extensible. You can also create custom data types, domains, and enumerated types to fit your business needs.

# NUMERIC TYPES

*PostgreSQL offers both fast approximate types and exact decimal types -- picking the wrong one silently corrupts money.*

| Type | Range / Precision | Use For |
|---|---|---|
| **SMALLINT** | -32,768 to 32,767 | Small counters, flags. |
| **INTEGER** | About -2.1B to 2.1B | Most whole-number columns, foreign keys. |
| **BIGINT** | About -9.2 quintillion to 9.2 quintillion | High-volume IDs, large counts. |
| **NUMERIC(p,s)** | Exact, user-defined precision/scale | Money, e.g. NUMERIC(19,2). |
| **REAL / DOUBLE PRECISION** | Approximate, floating-point | Scientific data -- never money. |

KEY TAKEAWAY  Use INTEGER or BIGINT for whole numbers and NUMERIC(p,s) for money -- never REAL or DOUBLE PRECISION, which are approximate and will silently round currency.

# PostgreSQL Numeric Types

PostgreSQL provides several numeric data types to store exact numbers, approximate numbers, and high-precision decimal values.

| Data Type | Aliases | Description | Storage / Size | Precision | Scale | Example Values |
|---|---|---|---|---|---|---|
| SMALLINT | int2 | Small-range integer | 2 bytes | Up to 5 decimal digits | 0 | -32768, 0, 32767 |
| INTEGER | int4 | Standard integer | 4 bytes | Up to 10 decimal digits | 0 | -2,147,483,648<br>0, 2,147,483,647 |
| BIGINT | int8 | Large-range integer | 8 bytes | Up to 19 decimal digits | 0 | -9,223,372,036,854,775,808<br>0, 9,223,372,036,854,775,807 |
| NUMERIC(p,s) | decimal | Exact numeric with user-defined precision and scale | Variable (up to 131,072 digits) | 1 – 131,072 | 0 – p | NUMERIC(10,2) = 12345.67<br>NUMERIC(20,6) = 123.456789 |
| DECIMAL(p,s) | decimal | Alias for NUMERIC(p,s) | Variable (up to 131,072 digits) | 1 – 131,072 | 0 – p | DECIMAL(10,2) = 12345.67 |
| REAL | float4 | Single-precision floating-point | 4 bytes | ≈ 6 – 7 decimal digits | N/A | 3.14159, -2.5, 1.0e6 |
| DOUBLE PRECISION | float8 | Double-precision floating-point | 8 bytes | ≈ 15 – 16 decimal digits | N/A | 3.141592653589793,<br>-1.234567890123456e10 |
| SERIAL | int4 (auto-increment) | Auto-incrementing integer (sequence-based) | 4 bytes | Up to 10 decimal digits | 0 | 1, 2, 3, 4, ... |
| BIGSERIAL | int8 (auto-increment) | Auto-incrementing big integer | 8 bytes | Up to 19 decimal digits | 0 | 1, 2, 3, 4, ... |

**Note:** Use INTEGER / BIGINT for whole numbers, NUMERIC / DECIMAL for exact decimal values, and REAL / DOUBLE PRECISION for approximate numeric values.

## Numeric Type Selection Guide

- Use **SMALLINT** for small ranges to save space.
- Use **INTEGER** for general-purpose whole numbers.
- Use **BIGINT** for very large whole numbers.
- Use **NUMERIC/DECIMAL** for financial, accounting, and high-precision data.
- Use **REAL** or **DOUBLE PRECISION** for scientific calculations where slight rounding is acceptable.

## Best Practices

- ✓ Use the smallest appropriate type to optimize storage and performance.
- ✓ Use **NUMERIC** with appropriate precision and scale for monetary values.
- ✓ Avoid floating-point types for financial calculations.
- ✓ Avoid over-allocating precision and scale.
- ✓ Use **BIGSERIAL** for large, auto-incrementing keys.

## Example Use Cases

- **SMALLINT:** Status codes, flags
- **INTEGER:** Age, quantity, count
- **NUMERIC(10,2):** Price, salary, invoice amount
- **DOUBLE PRECISION:** Scientific measurements
- **BIGSERIAL:** Primary key for large tables

# CHARACTER TYPES

*PostgreSQL's three character types behave almost identically -- the real-world default is VARCHAR or TEXT.*

| Type | Behavior | Recommendation |
|---|---|---|
| **VARCHAR(n)** | Variable length, up to n characters; rejects longer input. | Use when a meaningful maximum length exists, e.g. email VARCHAR(255). |
| **TEXT** | Variable length, unlimited. | Use for genuinely open-ended text, e.g. notes. |
| **CHAR(n)** | Fixed length; PostgreSQL pads shorter values with spaces. | Rarely needed -- reserve for fixed-width legacy codes. |

KEY TAKEAWAY  Prefer VARCHAR(n) when a real maximum length exists and TEXT when it doesn't -- PostgreSQL stores both identically, so there is no performance reason to prefer CHAR(n).

# PostgreSQL Character Types

Character types store textual data such as letters, numbers, symbols, and strings.
PostgreSQL provides both fixed-length and variable-length character data types.

| Data Type | Aliases | Description | Storage / Size | Example Value | Notes |
|---|---|---|---|---|---|
| **CHAR(n)** (character) | character(n) | Fixed-length character string. | Exactly n characters (padded with spaces if shorter) | `'ABC '` CHAR(5) | Use for values with constant length requirements. |
| **VARCHAR(n)** (character varying) | character varying(n), varchar(n) | Variable-length character string with maximum length n. | Up to n characters (only uses space needed) | `'Hello World'` VARCHAR(20) | Most common choice for general text data. |
| **TEXT** | text | Variable-length character string with no length limit. | Up to 1 GB of data | 'This is a text value of any length' | Use for large text documents or unlimited strings. |
| **CITEXT** | citext (extension) | Case-insensitive text type. | Up to 1 GB of data | 'Case Insensitive Text' citext | Case-insensitive comparisons and uniqueness. |
| **BPCHAR(n)** (blank padded) | bpchar(n) | Internal type used to store CHAR(n). | Exactly n characters (padded with spaces) | `'ABC '` BPCHAR(5) | System type mapped from CHAR(n). |
| **NAME** | name | Fixed-length string used for object names (identifiers). | Up to 63 bytes | `'customer_table'` NAME | Used internally for database object identifiers. |

## Key Takeaways

- Use **CHAR(n)** for fixed-length values.
- Use **VARCHAR(n)** for variable-length values with a defined limit.
- Use **TEXT** for large or unlimited-length strings.
- Use **CITEXT** for case-insensitive text comparisons.
- Names of database objects are stored using NAME type.

## Length Semantics

**CHAR(n)**

Always stores exactly n characters.

CHAR(5) = 'AB'
is stored as 'AB '
(3 trailing spaces)

**VARCHAR(n)**

Stores only the actual characters used.

VARCHAR(5) = 'AB'
is stored as 'AB'
(no extra spaces)

## Examples in Use

```
CREATE TABLE employee (
  emp_code      CHAR(6),          -- Fixed-length employee code
  emp_name      VARCHAR(100),     -- Variable-length name
  emp_bio       TEXT,             -- Long biography
  email         CITEXT            -- Case-insensitive email
);
```

```
SELECT 'abc' = 'ABC';                -- returns false
SELECT 'abc'::citext = 'ABC'::citext;   -- returns true
```

**Note:** CITEXT requires the extension:

```
CREATE EXTENSION IF NOT EXISTS citext;
```

## Best Practices

- Prefer **VARCHAR(n)** over **CHAR(n)** unless length is fixed.
- Use appropriate length n to balance storage and validation.
- Use **TEXT** for long descriptions, notes, or documents.
- Use **CITEXT** when user input should be case-insensitive.

# BOOLEAN TYPES

*PostgreSQL's BOOLEAN type has three possible states: true, false, and NULL (unknown).*

### TRUE

Accepts true, 't', 'yes', 'y', '1', and more as literal input.

### FALSE

Accepts false, 'f', 'no', 'n', '0', and more as literal input.

### NULL (Unknown)

A boolean column can be NULL unless declared NOT NULL -- 'unknown', not false.

### Use It Directly

Prefer BOOLEAN over an INTEGER or CHAR(1) flag column.

KEY TAKEAWAY  A PostgreSQL BOOLEAN is three-valued -- true, false, or NULL -- so is_active IS NULL is a real, different condition from is_active = false.

# PostgreSQL Boolean Types

Boolean types store true/false values and are commonly used for flags, status indicators, and conditional logic in PostgreSQL.

## Boolean Data Type

**BOOLEAN**

The BOOLEAN type stores logical values: `true` or `false`

| | |
|---|---|
| Internal Storage | 1 byte |
| Allowed Values | true, false |
| Default | false |
| Category | Boolean / Logical |
| Standard | SQL Standard compliant |

## Accepted Input Values

| Evaluated As TRUE | Evaluated As FALSE |
|---|---|
| true | false |
| t | f |
| yes | no |
| y | n |
| on | off |
| 1 | 0 |

Input is not case-sensitive. Extra spaces are ignored.

## Key Takeaways

- BOOLEAN stores `true` or `false` values.
- Commonly used for flags, switches, and status indicators.
- Use BOOLEAN for clear and efficient logical conditions.
- Default value is `false` if not specified.
- Boolean expressions in queries return `true` or `false`.

## Examples in Use

```
CREATE TABLE employee (
   id             SERIAL PRIMARY KEY,
   name           VARCHAR(100) NOT NULL,
   is_active      BOOLEAN NOT NULL DEFAULT true,
   email_verified BOOLEAN NOT NULL DEFAULT false
);

INSERT INTO employee (name, is_active, email_verified) VALUES
  ('Alice', true, true),
  ('Bob', false, false);
```

## Query Examples

```
-- Select active employees
SELECT * FROM employee WHERE is_active = true;

-- Select inactive employees
SELECT * FROM employee WHERE is_active = false;

-- Count verified emails
SELECT COUNT(*) FROM employee WHERE email_verified = true;

-- Boolean expression in SELECT
SELECT name, is_active, email_verified,
         (is_active AND email_verified) AS can_login
FROM employee;
```

## Best Practices

- Use BOOLEAN for logical values (true/false).
- Avoid storing boolean values as text or integers.
- Name boolean columns with prefixes like `is_`, `has_`, `can_`, `should_`, etc.
- Set appropriate default values for flags.
- Use boolean columns in WHERE, CHECK, and CASE expressions.

**Did You Know?** BOOLEAN is part of the SQL standard and evaluated as true (1) or false (0) in expressions and conditions.

# DATE AND TIME TYPES

*PostgreSQL's date/time types range from a plain calendar date to a fully timezone-aware timestamp.*

| Type | Stores | Recommendation |
|---|---|---|
| **DATE** | Calendar date only, no time. | Birthdays, due dates with no time component. |
| **TIME** | Time of day only, no date. | Rarely used alone -- daily schedules. |
| **TIMESTAMP** | Date and time, no timezone. | Avoid for anything crossing timezones. |
| **TIMESTAMPTZ** | Date and time, stored in UTC, timezone-aware. | Default choice for created_at, updated_at, and event times. |
| **INTERVAL** | A span of time, e.g. '3 days'. | Durations and date arithmetic results. |

KEY TAKEAWAY  Default to TIMESTAMPTZ for any column that records when something happened -- it stores every value normalized to UTC and converts correctly for any client's timezone.

# PostgreSQL Date and Time Types

PostgreSQL provides rich date and time types to store calendar dates, times of day, timestamps, and time intervals with high precision and time zone support.

## Core Date and Time Types

| Data Type | Description | Storage | Precision | Example Value | Notes |
|---|---|---|---|---|---|
| DATE | Calendar date (year, month, day) | 4 bytes | 1 day | 2025-05-18 | No time of day |
| TIME [(p)] | Time of day (without date) | 8 bytes | 1 microsecond (default p = 6) | 14:30:00.123456 | Range: 00:00:00 to 24:00:00 |
| TIME WITH TIME ZONE (TIMETZ) | Time of day with time zone offset | 12 bytes | 1 microsecond (default p = 6) | 14:30:00.123456+05:30 | Stores time and time zone offset |
| TIMESTAMP [(p)] (without time zone) | Date and time (without time zone) | 8 bytes | 1 microsecond (default p = 6) | 2025-05-18 14:30:00.123456 | Does not store time zone |
| TIMESTAMP WITH TIME ZONE (TIMESTAMPTZ) | Date and time with time zone | 8 bytes | 1 microsecond (default p = 6) | 2025-05-18 14:30:00.123456+05:30 | Stored in UTC, converted on retrieval |
| INTERVAL | Time interval (duration) | 16 bytes | 1 microsecond | 2 days 3 hours 15 minutes 10.123456 seconds | Can be positive or negative |

## Key Takeaways

- Use DATE for storing calendar dates.
- Use TIME for storing time of day.
- Use TIMESTAMP for date and time without time zone.
- Use TIMESTAMPTZ for date and time with time zone.
- PostgreSQL stores TIMESTAMPTZ in UTC internally.
- Use INTERVAL for durations and differences.

## Time Zone Behavior


TIMESTAMP: no time zone conversion is applied.
TIMESTAMPTZ: converted to/from session time zone.

## Example Usage

```
-- Insert examples
INSERT INTO event (event_date, start_time, created_at, duration)
VALUES
  ('2025-05-18', '14:30:00', '2025-05-18 14:30:00', INTERVAL '2 hours 15 minutes');

-- Current date and time examples
SELECT CURRENT_DATE;                    -- 2025-05-18
SELECT CURRENT_TIME;                    -- 14:30:00.123456
SELECT CURRENT_TIMESTAMP;               -- 2025-05-18 14:30:00.123456
SELECT CURRENT_TIMESTAMP AT TIME ZONE 'UTC';  -- 2025-05-18 14:30:00.123456+00
```

## Common Operations

| | |
|---|---|
| Add Interval | `timestamp + INTERVAL '1 day'` |
| Subtract Interval | `timestamp - INTERVAL '2 hours'` |
| Date Difference | `DATE '2025-05-18' - DATE '2025-05-10'` `-- returns 8 days` |
| Extract Part | `EXTRACT(YEAR FROM timestamp)` `EXTRACT(MONTH FROM date)` |
| Truncate | `DATE_TRUNC('day', timestamp)` |

## Best Practices

- Prefer TIMESTAMPTZ for any data that represents specific points in time.
- Store all timestamps in UTC; convert in the application layer.
- Be explicit about time zones in your applications.
- Use appropriate precision (p) only when needed.
- Avoid mixing TIMESTAMP and TIMESTAMPTZ in the same column for the same data.

PostgreSQL follows ISO 8601 standard for date and time formats. | Default timestamp precision is microsecond (6 digits). | Valid date range: 4713 BC to 5874897 AD.

# UUID

*A UUID is a 128-bit universally unique identifier -- ideal for public-facing IDs that must not reveal sequence or volume.*

### Globally Unique

Effectively collision-free across tables, databases, and even separate systems.

### Non-Sequential

Does not reveal how many rows exist or the order they were created in.

### Generated Two Ways

gen_random_uuid() in PostgreSQL, or generated in the application before insert.

### Trade-Off

Larger (16 bytes) than BIGINT and not naturally sortable by creation time.

KEY TAKEAWAY  Use UUID for identifiers exposed outside the database -- API responses, public URLs -- where an attacker or competitor should not be able to infer sequence or volume from the value.

# PostgreSQL UUID

UUID (Universally Unique Identifier) is a 128-bit identifier used to uniquely identify records across systems, databases, and distributed environments.

## What is UUID?

UUID is a 128-bit (16-byte) identifier that is designed to be globally unique.

- Extremely low probability of duplication
- Ideal for distributed systems
- Does not reveal information (non-sequential)
- Standard format defined by RFC 4122

### Example UUID

`550e8400-e29b-41d4-a716-446655440000`

| 8 hex digits | 4 hex digits | 4 hex digits | 4 hex digits | 12 hex digits |
|---|---|---|---|---|

## UUID in PostgreSQL

| | |
|---|---|
| Data Type | uuid |
| Storage Size | 16 bytes (128 bits) |
| Format | xxxxxxux-xxxx-xxxx-xxxx-xxxxxxuxxxxx |
| Case | Hex digits (0-9, a-f) |
| Hyphens | Displayed with hyphens (standard format) |
| Indexing | Fully indexable and efficient |

## Create Table Example

```
CREATE TABLE employee (
    id uuid PRIMARY KEY DEFAULT gen_random_uuid(),
  name  text NOT NULL,
  email text UNIQUE,
  created_at timestamp DEFAULT now()
);
```

## Key Takeaways

- UUID provides globally unique identifiers.
- Use `gen_random_uuid()` for random UUIDs in PostgreSQL.
- Ideal for microservices, replication, and data merging.
- UUIDs are non-sequential, improving security but affecting index locality.
- Store UUIDs using the `uuid` data type for best performance.

## Best Practices

- Use UUID as PRIMARY KEY in distributed systems.
- Use `gen_random_uuid()` for better performance and simplicity.
- Avoid mixing UUID and BIGSERIAL in the same column for same purpose.
- Consider index impact due to randomness.
- Validate UUID format in application layer if needed.

## Generating UUIDs

| | | |
|---|---|---|
| `gen_random_uuid()` | Generates a random UUID (requires `pgcrypto` extension) | `SELECT gen_random_uuid();` → `550e8400-e29b-41d4-a716-446655440000` |
| `uuid_generate_v4()` | Generates a random (v4) UUID (requires `uuid-ossp` extension) | `SELECT uuid_generate_v4();` → `f81d4fae-7dec-11d0-a765-00a0c91e6bf6` |
| `uuid_generate_v1mc()` | Generates time-based UUID (MAC address hidden) | `SELECT uuid_generate_v1mc();` → `a4c13c10-8f2e-11ee-be56-0242ac120002` |

## UUID Version Summary

| Version | Name | Description | Use Case |
|---|---|---|---|
| v1 | Time-based | Timestamp + MAC address | Legacy systems |
| v2 | DCE Security | Time-based with POSIX UID | Rarely used |
| v3 | Name-based (MD5) | Hash of name + namespace | Deterministic IDs |
| v4 | Random | Random (122 bits of randomness) | Most common |
| v5 | Name-based (SHA-1) | Hash of name + namespace | Deterministic IDs |

## Use Cases

- Distributed Systems
- Data Merging Across Systems
- Microservices Architecture
- Data Replication

Enable required extensions if needed:

```
CREATE EXTENSION IF NOT EXISTS "pgcrypto";  -- for gen_random_uuid()
CREATE EXTENSION IF NOT EXISTS "uuid-ossp";  -- for uuid_generate_*()
```

# JSON AND JSONB OVERVIEW

*PostgreSQL can store and query semi-structured JSON directly in a relational table -- JSONB is almost always the right choice.*

| Type | Storage | When To Use |
|---|---|---|
| **JSON** | Stored as exact input text; re-parsed on every read. | Rare -- only when preserving exact formatting/whitespace matters. |
| **JSONB** | Stored in a decomposed binary form; indexable and queryable. | The default choice for any JSON column -- faster to query, supports indexing. |

KEY TAKEAWAY  JSONB stores JSON in an efficient, indexable binary form and supports operators like -> and @> for querying inside the document -- use it for genuinely flexible data, not as an escape hatch from schema design.

# TRY IT YOURSELF — EXERCISE 1: CHOOSE COLUMN TYPES DELIBERATELY

*Reinforces: the PostgreSQL data type slides you just walked — numeric, character, boolean, date/time, UUID, and JSONB — applied to the CRM customer table.*

## STEPS (notes/lab37-type-choices.md)

| Step | What To Do |
|---|---|
| **1 — Column Table** | For each customer column, name the PostgreSQL type you would use. |
| **2 — Money Rule** | Say which numeric type holds a monetary balance, and which one must never. |
| **3 — Time Zones** | Choose between timestamp and timestamptz, and give the reason. |
| **4 — JSONB Limit** | Say when a JSONB column is appropriate and when a real column is better. |

## Type choices

```
id uuid | text 'CUS-1001'   name text   email text   status text
balance numeric(12,2)   -- never float/double for money
created_at timestamptz  -- absolute instant, not a wall-clock guess
```

TRY IT NOW — Complete Exercise 1 before continuing: exercises/exercise-01-type-choices.md (workspace: examples/module-37-exercises).

# PostgreSQL JSON and JSONB Overview

Store and query semi-structured data efficiently within PostgreSQL.
Choose between JSON (text) and JSONB (binary) based on your needs.

## JSON vs JSONB Comparison

| Feature | JSON (text) | JSONB (binary) |
|---|---|---|
| Storage Format | Stored as text | Stored in decomposed binary format |
| Parsing | Parsed on each read | Parsed on write, stored in binary |
| Performance | Slower for queries | Faster for queries and indexing |
| Indexing Support | Limited | Full GIN/GiST indexing support |
| Key Order | Preserves key order | Does not preserve key order |
| Duplicate Keys | Allowed | Last key wins (duplicates removed) |
| Whitespace | Preserved | Removed |
| Storage Size | Larger | Smaller (more efficient) |
| Use Case | Write-once, read-rarely, raw JSON | Read/Write worloarely, querying, indexing |

**Recommendation:** Use JSONB for most use cases due to better performance, indexing, and flexibility. Use JSON only when you need to preserve exact text and formatting.

## Example JSON Document

```
{
   "id": 101,
   "name": "Alice Johnson",
   "email": "alice.johnson@example.com",
   "active": true,
   "roles": ["admin", "user"],
   "address": {
      "street": "123 Main St",
      "city": "Dallas",
      "state": "TX",
      "zip": "75001"
   },
   "preferences": {
      "theme": "dark",
      "notifications": true
   },
   "createdAt": "2025-05-18T14:30:00Z"
}
```

## Key Takeaways

- JSON stores data as plain text; JSONB stores in binary format.
- JSONB is faster for querying, indexing, and filtering.
- JSONB supports GIN indexes and advanced operators.
- JSON preserves formatting, order, and duplicate keys.
- Use JSONB for most production applications.
- Both support flexible, schema-less data modeling.
- Combine JSONB with relational data for hybrid models.

## Common Use Cases

- User Profiles and Preferences
- Product Catalogs and Attributes
- IoT and Sensor Data
- Audit Logs and Events
- Config and Metadata

## Creating Table with JSONB

```
CREATE TABLE employee (
   id         SERIAL PRIMARY KEY,
   name       TEXT NOT NULL,
   details    JSONB,
   created_at TIMESTAMP DEFAULT now()
);
```

Use JSONB for indexing and efficient querying.

## Querying JSONB Data (Examples)

| | |
|---|---|
| Get value by key | `SELECT details->>'name' AS name FROM employee;` |
| Get nested value | `SELECT details->'address'->>'city' AS city FROM employee;` |
| Filter by key value | `SELECT * FROM employee WHERE details->>'active' = 'true';` |
| Check if key exists | `SELECT * FROM employee WHERE details ? 'roles';` |
| Containment | `SELECT * FROM employee WHERE details @> '{"roles": ["admin"]}';` |
| Array contains value | `SELECT * FROM employee WHERE details->'roles' ? 'admin';` |

## Indexing JSONB Data

```
-- Create GIN index for fast JSONB queries
CREATE INDEX idx_employee_details_gin .
  ON employee USING GIN (details);
```

Useful operators supported by JSONB:

| Operator | Description | | |
|---|---|---|---|
| -> | Get JSON object field | ? | Key exists |
| ->> | Get JSON object field as text | ?! | Any key exists in array |
| #> | Get JSON value by path | ?& | All keys exist in array |
| #>> | Get JSON value by path as text | @> | Contains |
| | | <@ | Is contained by |

**Best Practice:** Use JSONB for performance, indexing, and querying. Index frequently queried keys. Validate JSON structure at the application layer if needed.

Since PostgreSQL 9.2 (JSON) and 9.4 (JSONB)

# CREATING DATABASES

*CREATE DATABASE provisions a new, fully isolated database inside a PostgreSQL cluster.*

```
CREATE DATABASE crmdb
  WITH OWNER = crm_admin
       ENCODING = 'UTF8';

-- list databases in psql
\l
```

Run CREATE DATABASE while connected to any existing database (commonly postgres).
Always specify ENCODING = 'UTF8' explicitly -- don't rely on the cluster default.
DROP DATABASE permanently deletes it and every schema, table, and row inside -- no undo.
One database per application is the common default, e.g. crmdb for Northstar CRM.

KEY TAKEAWAY  CREATE DATABASE provisions a new, isolated database; run it once per application, always with an explicit UTF8 encoding, and treat DROP DATABASE as irreversible.

# PostgreSQL: Creating Databases

A database is a container for schemas, tables, and other database objects.
Use CREATE DATABASE to create a new database in PostgreSQL.

## Overview

Databases in PostgreSQL are independent of each other.
Each database has its own:

- Schemas
- Tables and objects
- Users and permissions
- Configuration settings
- Storage (files on disk)

## Syntax

```
CREATE DATABASE database_name
  [ WITH
    OWNER = owner_name
    ENCODING = 'UTF8'
    LC_COLLATE = 'en_US.UTF-8'
    LC_CTYPE   = 'en_US.UTF-8'
    TABLESPACE = tablespace_name
    CONNECTION LIMIT = -1
    IS_TEMPLATE = template_name ];
```

## Examples

**1 Create a simple database**

```
CREATE DATABASE employee_db;
```

Creates a database with default settings.

**2 Create a database with owner**

```
CREATE DATABASE employee_db
  WITH OWNER = app_user;
```

Sets the owner of the database.

**3 Create a database with encoding and locale**

```
CREATE DATABASE reporting_db
  WITH ENCODING = 'UTF8'
       LC_COLLATE = 'en_US.UTF-8'
       LC_CTYPE   = 'en_US.UTF-8';
```

Specifies encoding and locale for the database.

**4 Create a database with tablespace and connection limit**

```
CREATE DATABASE analytics_db
  WITH TABLESPACE  = fastspace
       CONNECTION LIMIT = 50;
```

Stores data in the specified tablespace and limits concurrent connections.

**5 Create a database from a template**

```
CREATE DATABASE dev_db
  WITH TEMPLATE  template0
       OWNER = dev_user;
```

Creates a database using an existing template.

## Prerequisites

- You must have the **CREATEDB** privilege.
- You must **connect to a database other than** the one you are creating.
- Use a **superuser account** or a user with required permissions.

## Best Practices

- Use meaningful and consistent database names.
- Assign a dedicated owner for each database.
- Choose the correct encoding and locale based on application needs.
- Use appropriate tablespaces for performance and storage management.
- Set connection limits to protect the system from too many connections.
- Regularly back up important databases.

## Common Templates

| Template | Description |
|---|---|
| template0 | Empty template with no objects or settings. |
| template1 | Default template with standard objects and settings. |
| Custom Template | A copy of an existing database for consistent setup. |

## List Existing Databases

List all databases in the cluster:

```
\l
```

Or using SQL:

```
SELECT datname, owner, encoding, datcollate, datctype
FROM pg_database
ORDER BY datname;
```

## Connect to the New Database

Connect to the newly created database:

```
\c employee_db
```

Verify the current database:

```
SELECT current_database();
```

**Note:** Once created, you can manage schemas, tables, users, permissions, and other objects within the database.

Databases are stored in the data directory on disk as separate folders. Deleting a database removes all its objects permanently.

Be careful when dropping databases – there is no UNDO.

# CREATING SCHEMAS

*CREATE SCHEMA adds a named namespace inside the current database for grouping related tables.*

```
CREATE SCHEMA crm AUTHORIZATION crm_app;

-- qualify a table with its schema
SELECT * FROM crm.customer;

-- or set a default search path
SET search_path TO crm, public;
```

Every database starts with a default schema named public.
search_path controls which schema an unqualified table name resolves to.
Schemas group tables logically -- by module, feature area, or tenant.
For Lab 37, the default public schema is enough -- extra schemas are optional polish.

KEY TAKEAWAY  A schema is a namespace inside a database, resolved through search_path -- most applications, including Lab 37, can stay entirely in the default public schema.

# Creating Schemas

Schemas help organize database objects into logical groups within a database. The **public** schema is created by default, but you can create custom schemas to support modular, secure, and maintainable database design.

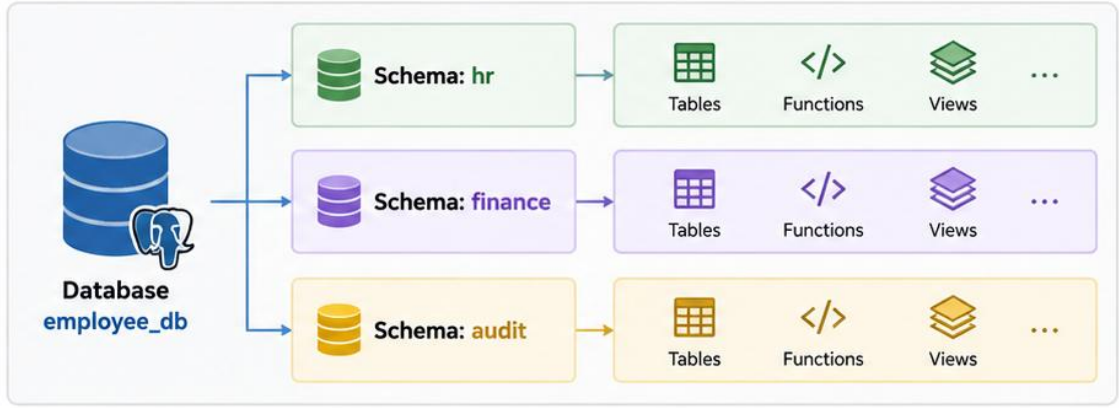
### Logical Organization
Group related tables, views, functions, and other objects under a common namespace.

### Security & Access Control
Grant permissions at the schema level to control access for users and roles.

### Modularity
Isolate applications or business domains in separate schemas within the same database.

### Manageability
Simplifies maintenance, deployment, and collaboration across teams.

**Database → Schemas → Objects**




**SQL Example: Create a Schema**

```sql
-- Create a new schema
CREATE SCHEMA hr;

-- Create a schema with authorization
CREATE SCHEMA finance AUTHORIZATION app_user;

-- List all schemas in the database
SELECT schema_name FROM information_schema.schemata;
```

The **public** schema is available by default.

You can set your **search_path** to **control** which schemas are used by default.

**Key Takeaway:** Schemas provide structure, security, and scalability within a database. Use schemas to organize objects, enforce access control, and support modular application design.

# CREATING TABLES

*CREATE TABLE defines a table's columns, types, and constraints in one statement.*

```
CREATE TABLE customer (
  customer_id  BIGINT GENERATED ALWAYS AS IDENTITY PRIMARY KEY,
  full_name    VARCHAR(150) NOT NULL,
  email        VARCHAR(255) NOT NULL UNIQUE,
  status       VARCHAR(20)  NOT NULL DEFAULT 'PROSPECT',
  created_at   TIMESTAMPTZ  NOT NULL DEFAULT now()
);
```

KEY TAKEAWAY  CREATE TABLE declares every column's name, type, and constraints together -- the primary key, NOT NULL, UNIQUE, and DEFAULT clauses shown here are each covered in depth next.

# Creating Tables

Tables store data in rows and columns. A well-designed table structure ensures data integrity, performance, and scalability. Define columns, data types, and constraints to enforce business rules at the database level.

### Structured Storage
Organize data in tables with rows (records) and columns (fields).

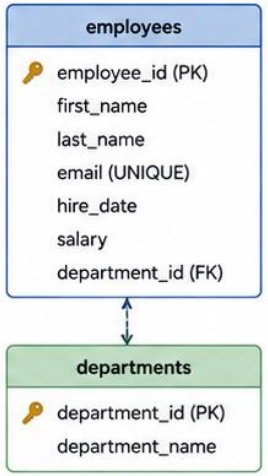
### Data Integrity
Use constraints like **PRIMARY KEY**, **FOREIGN KEY**, **NOT NULL**, **UNIQUE**, and **CHECK** to maintain accurate and consistent data.

### Performance
Choose appropriate data types and indexing strategies to optimize query performance.

### Scalability & Maintainability
A clear table design supports future growth, easier maintenance, and team collaboration.

**Example Table: employees**

| Column Name | Data Type | Constraints | Description |
|---|---|---|---|
| employee_id | SERIAL | PRIMARY KEY | Unique identifier for employee |
| first_name | VARCHAR(50) | NOT NULL | Employee first name |
| last_name | VARCHAR(50) | NOT NULL | Employee last name |
| email | VARCHAR(100) | UNIQUE, NOT NULL | Employee email address |
| hire_date | DATE | NOT NULL | Date of joining |
| salary | NUMERIC(12,2) | CHECK (salary > 0) | Monthly salary |
| department_id | INTEGER | FOREIGN KEY | Reference to departments table |




**SQL Example: Create Table**

```sql
-- Create employees table
CREATE TABLE employees (
  employee_id   SERIAL PRIMARY KEY,
  first_name    VARCHAR(50) NOT NULL,
  last_name     VARCHAR(50) NOT NULL,
  email         VARCHAR(100) NOT NULL UNIQUE,
  hire_date     DATE NOT NULL,
  salary        NUMERIC(12,2) CHECK (salary > 0),
  department_id INTEGER REFERENCES departments(department_id)
);

```

### Best Practices
- Use meaningful and consistent table and column names.
- Choose the right data types.
- Define primary keys for every table.
- Apply constraints to enforce business rules.
- Index foreign keys for better performance.

**Key Takeaway:** A well-designed table structure is the foundation of a strong database. It ensures data integrity, enforces business rules, and supports high-performance applications.

# PRIMARY KEYS

*A primary key is a column, or set of columns, that uniquely identifies every row in a table.*

### Must Be Unique

No two rows can ever share the same primary key value.

### Cannot Be NULL

Every row must have a real value -- PostgreSQL enforces this automatically.

### Only One Per Table

A table can declare exactly one primary key, single or composite.

### Anchors Foreign Keys

Other tables reference this table safely by pointing at its primary key.

KEY TAKEAWAY  A primary key is always unique and never NULL, and every foreign key relationship this module covers depends on one existing to point at.

# Primary Keys

A **Primary Key** is a column or a set of columns that **uniquely identifies** each row in a table.

## Key Characteristics

### Uniqueness
Each value in the primary key must be unique.

### NOT NULL
Primary key columns cannot contain NULL values.

### Entity Integrity
Ensures each row can be reliably identified and referenced.

### Referencing
Referenced by foreign keys in other tables to establish relationships.

## Example Table: employees

| employee_id (PK) | first_name | last_name | email | hire_date | salary |
|---|---|---|---|---|---|
| 1 | John | Doe | john.doe@example.com | 2024-01-15 | 65000.00 |
| 2 | Jane | Smith | jane.smith@example.com | 2024-02-10 | 72000.00 |
| 3 | Michael | Brown | michael.brown@example.com | 2024-03-05 | 68000.00 |
| 4 | Emily | Johnson | emily.johnson@example.com | 2024-04-20 | 70000.00 |

**Unique identifier** for each employee

**Duplicate values are not allowed**

## Why Primary Keys Matter

- Guarantees row uniqueness
- Enables reliable relationships
- Improves data consistency
- Optimizes query performance

## SQL Examples

### 1. Define Primary Key Inline

```sql
CREATE TABLE employees (
  employee_id SERIAL PRIMARY KEY,
  first_name VARCHAR(50) NOT NULL,
  last_name VARCHAR(50) NOT NULL,
  email VARCHAR(100) UNIQUE,
  hire_date DATE NOT NULL
);
```

### 2. Define Primary Key as a Table Constraint

```sql
CREATE TABLE employees (
  employee_id INT,
  first_name VARCHAR(50) NOT NULL,
  last_name VARCHAR(50) NOT NULL,
  email VARCHAR(100) UNIQUE,
  hire_date DATE NOT NULL,
  PRIMARY KEY (employee_id)
);
```

## Best Practices

- Use surrogate keys (e.g., SERIAL, UUID) instead of natural keys.
- Keep primary keys simple and immutable.
- Use meaningful names like `id`, `user_id`, `order_id`.
- Avoid updating primary key values.

**Key Takeaway:** Primary keys uniquely identify each row, enforce data integrity, and serve as the foundation for relationships and efficient database design.

# FOREIGN KEYS

*A foreign key is a column in one table that must reference an existing primary key value in another table.*

```
CREATE TABLE account (
  account_id   BIGINT GENERATED ALWAYS AS IDENTITY PRIMARY KEY,
  customer_id  BIGINT NOT NULL,
  balance      NUMERIC(19,2) NOT NULL DEFAULT 0,
  CONSTRAINT fk_account_customer
    FOREIGN KEY (customer_id) REFERENCES customer(customer_id)
);
```

Every customer_id in account must already exist in customer.customer_id, or be NULL.
Inserting account with a non-existent customer_id is rejected -- no orphan rows.
A foreign key can repeat (many accounts per customer) -- uniqueness is not required.

KEY TAKEAWAY  A foreign key enforces referential integrity automatically -- account.customer_id can only hold a value that genuinely exists in customer.customer_id, or NULL.

# Foreign Keys

A **Foreign Key** is a column (or set of columns) in one table that refers to the **Primary Key** of another table.

## Purpose

Foreign keys enforce **referential integrity** by ensuring that related data between tables remains consistent.

## Key Benefits

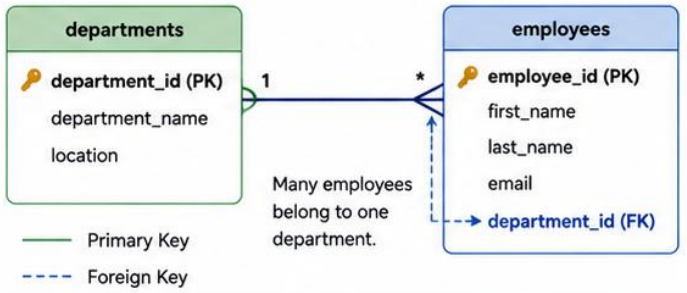
**Data Integrity**
Prevents orphan records and maintains valid relationships.

**Consistency**
Ensures that related data exists before it is referenced.

**Relationship Management**
Supports one-to-many, many-to-many, and other relationships.

**Reliable Operations**
Simplifies joins and improves trust in data accuracy.

## Example Relationship: Employees and Departments




## SQL Example: Creating a Foreign Key

```sql
-- Parent table: departments
CREATE TABLE departments (
    department_id SERIAL PRIMARY KEY,
    department_name VARCHAR(100) NOT NULL,
    location VARCHAR(100)
);

-- Child table: employees with foreign key
CREATE TABLE employees (
    employee_id SERIAL PRIMARY KEY,
    first_name VARCHAR(50) NOT NULL,
    last_name VARCHAR(50) NOT NULL,
    email VARCHAR(100) UNIQUE,
    department_id INT NOT NULL,
    CONSTRAINT fk_employees_department
        FOREIGN KEY (department_id)
        REFERENCES departments(department_id)
        ON UPDATE CASCADE
        ON DELETE RESTRICT
);
```

### Common Actions

**ON DELETE CASCADE**
Automatically deletes child rows when parent is deleted.

**ON DELETE RESTRICT**
Prevents deletion of parent if child rows exist.

**ON UPDATE CASCADE**
Updates child rows when parent key is updated.

## Sample Data

**departments**

| department_id (PK) | department_name | location |
|---|---|---|
| 1 | Human Resources | New York |
| 2 | Engineering | Austin |
| 3 | Finance | Chicago |

**employees**

| employee_id (PK) | first_name | last_name | department_id (FK) |
|---|---|---|---|
| 101 | John | Doe | 2 |
| 102 | Jane | Smith | 2 |
| 103 | Emily | Johnson | 1 |
| 104 | Michael | Brown | 3 |

If a department is deleted, employees referencing that department cannot remain (depending on the **ON DELETE** rule).

## Best Practices

- Always define foreign keys to enforce relationships.
- Choose appropriate **ON DELETE** and **ON UPDATE** actions.
- Index foreign key columns for better performance.
- Use meaningful constraint names.
- Design relationships carefully before enforcing constraints.

**Key Takeaway:** Foreign keys connect tables and enforce referential integrity. They ensure that relationships between data remain valid, consistent, and reliable.

# UNIQUE CONSTRAINTS

*A UNIQUE constraint ensures every value in a column, or set of columns, is distinct across the whole table.*

```
email VARCHAR(255) NOT NULL UNIQUE,
CONSTRAINT uk_customer_email UNIQUE (email)
```

A table can have many UNIQUE constraints, but only one PRIMARY KEY.
A nullable UNIQUE column allows at most one NULL in most configurations.
A duplicate insert fails with: duplicate key value violates unique constraint.
Typical candidates: email, username, and any other real-world 'must be distinct' value.

KEY TAKEAWAY  UNIQUE guarantees distinct values but, unlike a primary key, a table can have several UNIQUE constraints and the column may still allow NULL.

# Unique Constraints

A **UNIQUE** constraint ensures that all values in a column (or set of columns) are distinct across all rows in a table.

## Key Benefits

### Prevents Duplicate Data
Ensures no two rows have the same value(s) in the constrained column(s).

### Enforces Business Rules
Useful for columns like email, username, code, or any attribute that must be distinct.

### Improves Data Quality
Helps maintain accuracy and reliability of database records.

### Supports Performance
Unique constraints are automatically indexed by PostgreSQL for fast lookups.

## Example: Employees Table

| Column | Data Type | Constraints | Description |
|---|---|---|---|
| employee_id | SERIAL | PRIMARY KEY | Unique identifier for employee |
| first_name | VARCHAR(50) | NOT NULL | Employee first name |
| last_name | VARCHAR(50) | NOT NULL | Employee last name |
| email | VARCHAR(100) | UNIQUE, NOT NULL | Employee email (must be unique) |
| phone_number | VARCHAR(20) | UNIQUE | Employee phone (must be unique) |
| hire_date | DATE | NOT NULL | Date of joining |

## Sample Data (emails must be unique)

| employee_id | first_name | email |
|---|---|---|
| 1 | John | john.doe@example.com |
| 2 | Jane | jane.smith@example.com |
| 3 | Michael | michael.brown@example.com |
| 4 | Emily | emily.johnson@example.com |

### Violation Example
Inserting another row with email = jane.smith@example.com will fail.

## SQL Examples

```sql
-- 1. Defining a UNIQUE constraint inline
CREATE TABLE employees (
    employee_id SERIAL PRIMARY KEY,
    first_name  VARCHAR(50) NOT NULL,
    last_name   VARCHAR(50) NOT NULL,
    email       VARCHAR(100) NOT NULL UNIQUE,
    phone_number VARCHAR(20) UNIQUE,
    hire_date   DATE NOT NULL
);

-- 2. Adding a UNIQUE constraint to an existing table
ALTER TABLE employees
   ADD CONSTRAINT uq_employees_email UNIQUE (email);
```

### Notes
- Multiple **UNIQUE** constraints can exist in a table.
- **UNIQUE** allows **one NULL** value (or multiple **NULLs** in PostgreSQL).
- For multiple columns, use a **combined UNIQUE** constraint.

## Best Practices
- Use meaningful, business-specific column names.
- Apply **UNIQUE** constraints to natural keys (e.g., email, code).
- Prefer **UNIQUE** over application-level checks.
- Consider case-insensitive uniqueness (using citext extension if needed).
- Combine with **NOT NULL** unless NULLs are intentionally allowed.

**Key Takeaway:** Unique constraints ensure that data remains distinct and reliable. Use them to enforce business rules, prevent duplicates, and improve overall data integrity.

# NOT NULL CONSTRAINTS

*A NOT NULL constraint ensures a column can never hold NULL -- it makes a value mandatory.*

| Column | Constraint | Why |
|---|---|---|
| **full_name** | NOT NULL | A customer record without a name is not usable. |
| **email** | NOT NULL | Required for every notification and login flow. |
| **middle_name** | nullable (no NOT NULL) | Genuinely optional -- not every customer has one. |
| **closed_at** | nullable (no NOT NULL) | Only set once an account is actually closed. |

KEY TAKEAWAY  Default to NOT NULL for every column a business rule truly requires, and leave a column nullable only when the absence of a value is itself a meaningful, expected state.

# 24. NOT NULL Constraints

## NOT NULL Constraints

A **NOT NULL** constraint prevents a column from storing **NULL** values. It ensures that every row must have a value in the constrained column(s).

### Key Benefits

**Ensures Mandatory Data**
Guarantees that important columns always contain a value.

**Improves Data Integrity**
Reduces incomplete or missing information in the database.

**Supports Business Rules**
Enforces rules that certain fields are required for valid records.

**Better Application Reliability**
Applications can rely on the presence of data without additional **NULL** checks.

### Example: Employees Table

| Column Name | Data Type | Constraints | Description |
|---|---|---|---|
| employee_id | SERIAL | PRIMARY KEY | Unique identifier |
| first_name | VARCHAR(50) | NOT NULL | Employee first name (required) |
| last_name | VARCHAR(50) | NOT NULL | Employee last name (required) |
| email | VARCHAR(100) | UNIQUE, NOT NULL | Employee email (required & unique) |
| phone_number | VARCHAR(20) | NULL | Employee phone (optional) |
| hire_date | DATE | NOT NULL | Date of joining (required) |
| salary | NUMERIC(12,2) | NOT NULL | Salary (required) |

### Sample Data (NOT NULL enforced)

| employee_id | first_name | last_name | email |
|---|---|---|---|
| 1 | John | Doe | john.doe@example.com |
| 2 | Jane | Smith | jane.smith@example.com |
| 3 | Michael | Brown | michael.brown@example.com |

### What Happens if NOT NULL is Violated?

Inserting a row with **NULL** in a **NOT NULL** column will result in an error.

```
ERROR:  null value in column "email" of relation
"employees" violates not-null constraint
```

### SQL Examples

**1. Defining NOT NULL in Table Creation**

```sql
CREATE TABLE employees (
    employee_id  SERIAL PRIMARY KEY,
    first_name   VARCHAR(50) NOT NULL,
    last_name    VARCHAR(50) NOT NULL,
    email        VARCHAR(100) NOT NULL,
    hire_date    DATE NOT NULL,
    salary       NUMERIC(12,2) NOT NULL,
    phone_number VARCHAR(20)
);
```

**2. Adding NOT NULL to Existing Column**

```sql
ALTER TABLE employees
 ALTER COLUMN hire_date SET NOT NULL;
```

You cannot add NOT NULL if any existing row contains NULL in that column. Update or remove NULLs before applying.

### Best Practices

- Apply **NOT NULL** to all required business-critical fields.
- Use **NOT NULL** along with appropriate data types, constraints, and defaults.
- Avoid allowing **NULLs** unless the data is truly optional.
- Validate data at the application level to prevent constraint violations.
- Review and clean existing data before adding **NOT NULL** constraints.

**Key Takeaway:** NOT NULL constraints ensure that columns always contain meaningful data. They help maintain data quality, enforce business rules, and make applications more reliable.

# CHECK CONSTRAINTS

*A CHECK constraint restricts the values a column can hold by enforcing a boolean condition.*

```
CREATE TABLE customer (
  customer_id BIGINT GENERATED ALWAYS AS IDENTITY PRIMARY KEY,
  status      VARCHAR(20) NOT NULL DEFAULT 'PROSPECT',
  CONSTRAINT ck_customer_status
    CHECK (status IN ('PROSPECT','ACTIVE','SUSPENDED','CLOSED'))
);
```

KEY TAKEAWAY  A CHECK constraint enforces a business rule directly in the database -- an insert or update that violates it is rejected before the row ever reaches storage.

# CHECK Constraints

A CHECK constraint ensures that all values in a column (or set of columns) satisfy a specific condition or rule.

## Key Benefits

### Enforces Business Rules
Validates data based on application logic (e.g., salary must be greater than 0).

### Improves Data Integrity
Prevents invalid or out-of-range data from being stored.

### Adds Flexibility
Allows complex validations beyond data types, NOT NULL, or UNIQUE constraints.

### Catches Errors Early
Ensures only valid data enters the database, reducing downstream issues.

## Example: Employees Table with CHECK Constraints

| Column Name | Data Type | Constraints | Description | Example Rule |
|---|---|---|---|---|
| employee_id | SERIAL | PRIMARY KEY | Unique identifier | – |
| first_name | VARCHAR(50) | NOT NULL | Employee first name | – |
| last_name | VARCHAR(50) | NOT NULL | Employee last name | – |
| email | VARCHAR(100) | UNIQUE, NOT NULL | Employee email | – |
| salary | NUMERIC(12,2) | NOT NULL, CHECK (salary > 0) | Salary must be greater than 0 | salary > 0 |
| hire_date | DATE | NOT NULL, CHECK (hire_date <= CURRENT_DATE) | Hire date cannot be in the future | hire_date <= CURRENT_DATE |
| department_id | INT | NOT NULL, CHECK (department_id > 0) | Department id must be a positive number | department_id > 0 |
| status | VARCHAR(20) | NOT NULL, CHECK (status IN ('ACTIVE','INACTIVE')) | Status must be one of the allowed values | status IN ('ACTIVE','INACTIVE') |

## SQL Examples

1. Creating Table with CHECK Constraints

```sql
CREATE TABLE employees (
  employee_id SERIAL PRIMARY KEY,
  first_name  VARCHAR(50) NOT NULL,
  last_name   VARCHAR(50) NOT NULL,
  email   VARCHAR(100) UNIQUE NOT NULL,
  salary  NUMERIC(12,2) NOT NULL
    CHECK (salary > 0),
  hire_date  DATE NOT NULL
    CHECK (hire_date <= CURRENT_DATE),
  department_id INT NOT NULL
    CHECK (department_id > 0),
  status  VARCHAR(20) NOT NULL
    CHECK (status IN ('ACTIVE','INACTIVE'))
);
```

2. Adding a CHECK Constraint to Existing Table

```sql
ALTER TABLE employees
 ADD CONSTRAINT chk_salary_positive
CHECK (salary > 0);
```

3. Dropping a CHECK Constraint

```sql
ALTER TABLE employees
DROP CONSTRAINT chk_salary_positive;
```

CHECK constraints are enforced per row and cannot reference other rows.

They help keep data accurate and consistent based on defined business rules.

## Sample Data

| employee_id | salary | hire_date | status |
|---|---|---|---|
| 1 | 75000.00 | 2023-05-10 | ACTIVE |
| 2 | 62000.00 | 2022-11-01 | ACTIVE |
| 3 | 58000.00 | 2024-03-20 | INACTIVE |

## Constraint Violation Examples

- salary = -5000 → Violates: salary > 0
- hire_date = '2026-01-01' → Violates: hire_date <= CURRENT_DATE
- status = 'PENDING' → Violates: status IN ('ACTIVE','INACTIVE')
- department_id = 0 → Violates: department_id > 0

## Best Practices

- Keep CHECK conditions simple and clear.
- Use meaningful names for constraints.
- Validate rules at the database level to protect data integrity.
- Test constraints with valid and invalid data.
- Avoid heavy computations or subqueries in CHECK conditions.

**Key Takeaway:** CHECK constraints enforce domain-specific business rules and ensure that only valid, meaningful data is stored in your PostgreSQL database.

# DEFAULT VALUES

*A DEFAULT clause supplies a column's value automatically whenever an INSERT does not specify one.*

| Column | Default | Behavior |
|---|---|---|
| **status** | DEFAULT 'PROSPECT' | New customers start as PROSPECT unless stated otherwise. |
| **created_at** | DEFAULT now() | Stamped with the current instant automatically. |
| **balance** | DEFAULT 0 | New accounts start at zero, never left NULL. |
| **customer_id** | GENERATED ALWAYS AS IDENTITY | A generated value, conceptually similar to a default. |

KEY TAKEAWAY  A DEFAULT is applied only when a column is omitted from an INSERT entirely -- it never overrides a value the statement actually supplies, including an explicit NULL.

# TRY IT YOURSELF — EXERCISE 2: PLAN KEYS AND CONSTRAINTS

*Reinforces: creating tables, primary and foreign keys, unique, not null, check constraints, and defaults as you just covered them.*

## STEPS (notes/lab37-constraints.md)

| Step | What To Do |
|---|---|
| **1 — Key Choice** | Name the primary key for customer and account, and say why. |
| **2 — Constraint List** | Give one unique, one not-null, and one check constraint for the CRM. |
| **3 — Foreign Key** | Write the account-to-customer foreign key and its delete behaviour. |
| **4 — Why in the Database** | Say why these rules belong in the schema and not only in Java. |

## CRM constraints

```
customer(id pk)   account(id pk, customer_id references customer(id))
unique(email)   not null(name, status)
check (status in ('ACTIVE','PROSPECT'))   -- the database is the last line
```

TRY IT NOW — Complete Exercise 2 before continuing: exercises/exercise-02-constraint-plan.md (workspace: examples/module-37-exercises).

# DEFAULT Values

A DEFAULT value is used when no value is provided for a column during INSERT. The database automatically assigns the default value defined for that column.

## Key Benefits

**Ensures Consistency**
Applies a standard value when none is provided, keeping data consistent.

**Reduces Errors**
Prevents NULL or missing values for important columns.

**Improves Productivity**
Simplifies INSERT operations by reducing the need to specify values for every column.

**Supports Business Rules**
Helps enforce default business logic (e.g., status = 'ACTIVE').

## Example: Employees Table with DEFAULT Values

| Column Name | Data Type | Constraints / Default | Description | Example Default |
|---|---|---|---|---|
| employee_id | SERIAL | PRIMARY KEY | Unique identifier | – |
| first_name | VARCHAR(50) | NOT NULL | Employee first name | – |
| last_name | VARCHAR(50) | NOT NULL | Employee last name | – |
| email | VARCHAR(100) | UNIQUE, NOT NULL | Employee email | – |
| status | VARCHAR(20) | NOT NULL DEFAULT 'ACTIVE' | Employment status | 'ACTIVE' |
| hire_date | DATE | NOT NULL DEFAULT CURRENT_DATE | Date of joining | Current date |
| created_at | TIMESTAMP | NOT NULL DEFAULT CURRENT_TIMESTAMP | Record creation time | Current timestamp |
| salary | NUMERIC(12,2) | NOT NULL DEFAULT 0 | Employee salary | 0 |

## SQL Examples

1. Creating Table with DEFAULT Values

```
CREATE TABLE employees (
  employee_id SERIAL PRIMARY KEY,
  first_name VARCHAR(50) NOT NULL,
  last_name  VARCHAR(50) NOT NULL,
  email  VARCHAR(100) UNIQUE NOT NULL,
  status VARCHAR(20) NOT NULL DEFAULT 'ACTIVE',
  hire_date DATE NOT NULL DEFAULT CURRENT_DATE,
  created_at TIMESTAMP NOT NULL
  DEFAULT CURRENT_TIMESTAMP,
  salary NUMERIC(12,2) NOT NULL DEFAULT 0
);
```

2. Altering Table to Add a DEFAULT

```
ALTER TABLE employees
  ALTER COLUMN status SET DEFAULT 'ACTIVE';

ALTER TABLE employees
  ALTER COLUMN salary SET DEFAULT 0;
```

DEFAULT values are applied only when a value is not provided in the INSERT statement.

## Sample Inserts and Results

**1. Insert without specifying default columns**

```
INSERT INTO employees (first_name, last_name, email)
VALUES ('John', 'Doe', 'john.doe@example.com');
```

| status | hire_date | created_at | salary |
|---|---|---|---|
| 'ACTIVE' | 2025-05-11 | 2025-05-11 10:30:45 | 0.00 |

**2. Insert with custom values (overrides defaults)**

```
INSERT INTO employees (first_name, last_name, email,
status, salary)
VALUES ('Jane', 'Smith', 'jane.smith@example.com',
'INACTIVE', 75000.00);
```

| status | hire_date | created_at | salary |
|---|---|---|---|
| 'INACTIVE' | 2025-05-11 | 2025-05-11 10:30:60 | 75000.00 |

## Best Practices

- Use DEFAULT for values that are common or expected.
- Use meaningful defaults (e.g., CURRENT_DATE, 0, 'ACTIVE').
- Avoid hiding data issues behind defaults.
- Review defaults carefully before applying to production.
- Document default values for clarity.

**Key Takeaway:** DEFAULT values help ensure data completeness, maintain consistency, and simplify data entry by automatically providing values when none are supplied.

# TABLE RELATIONSHIPS

*A relationship defines how two or more tables are associated -- the core of relational schema design.*

| Cardinality | Notation | Enforced By |
|---|---|---|
| **One-to-One** | 1 : 1 | Foreign key with a UNIQUE constraint on the child. |
| **One-to-Many** | 1 : M | Foreign key on the 'many' side referencing the 'one' side. |
| **Many-to-Many** | M : M | A junction table with foreign keys to both sides. |

KEY TAKEAWAY  Every relationship in a relational schema reduces to one of three cardinalities, and each is implemented with foreign keys -- the next slide walks a worked example of each.

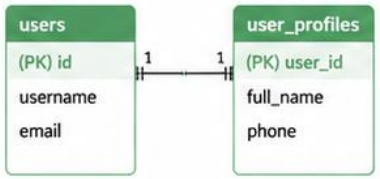
37 **MODULE 37:** PostgreSQL Database Fundamentals

# Table Relationships

Relationships define how data in one table connects to data in another.

## 1 One-to-One

A record in Table A relates to one and only one record in Table B.

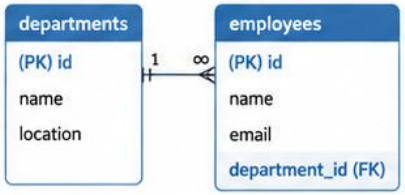



users.id = user_profiles.user_id

- Used when additional details are stored separately.
- Example: users and user_profiles
- Implement with UNIQUE foreign key in one of ten tables.

## 2 One-to-Many

A record in Table A relates to many records in Table B.

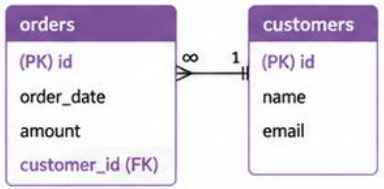



employees.department_id → departments.id

- Most common relationship.
- Example: departments and employees
- Foreign key in the "many" table references the "one" table.

## 3 Many-to-One

Many records in Table A relate to one record in Table B.

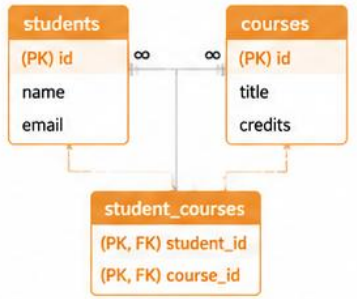



orders.customer_id → customers.id

- A variation of one-to-many.
- Example: orders and customers
- Many orders belong to one customer.

## 4 Many-to-Many

Many records in Table A relate to many records in Table B.




- Requires a join table.
- Example: students and courses
- Join table contains foreign keys to both tables.

## Key Points

- Relationships enforce data integrity.
- Use foreign keys to maintain valid references.
- Proper relationships improve query performance.
- Choose the right relationship to match business needs.
- Well-designed relationships support scalability.

**KEY TAKEAWAY:** Table relationships model real-world connections, enforce data integrity, and enable powerful queries.

# ONE-TO-ONE, ONE-TO-MANY, MANY-TO-MANY

*Three worked examples from the Northstar CRM schema -- one for each cardinality.*

### 1 One-to-One

customer <-> customer_profile. A UNIQUE foreign key on customer_profile.customer_id guarantees at most one profile per customer.

### One-to-Many

customer -> account. A plain foreign key, account.customer_id, lets one customer own many accounts.

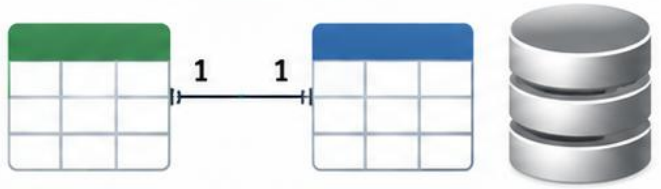
### Many-to-Many

account <-> tag, resolved by a junction table account_tag(account_id, tag_id) with a foreign key to each side.

KEY TAKEAWAY  One-to-one adds UNIQUE to the foreign key, one-to-many uses a plain foreign key on the many side, and many-to-many always needs a junction table -- there is no fourth pattern.

# One-to-One

A record in Table A relates to one and only one record in Table B, and vice versa.




## EXAMPLE

Each user has exactly one user profile, and each user profile belongs to exactly one user.

| users | |
|---|---|
| id | PK |
| username | VARCHAR |
| email | VARCHAR |
| created_at | TIMESTAMP |

1 — 1

| user_profiles | |
|---|---|
| user_id | PK, FK |
| full_name | VARCHAR |
| phone | VARCHAR |
| address | VARCHAR |
| date_of_birth | DATE |

**users.id = user_profiles.user_id**
The foreign key in user_profiles is also the primary key, ensuring one-to-one relationship.

## KEY CHARACTERISTIC

**1:1** One record in Table A relates to one record in Table B.

**Bidirectional**
The relationship is mutual.

**Enforces Uniqueness**
The foreign key must be UNIQUE (or the primary key).

**Used for Separate Details**
Useful when additional details are stored in a separate table.

**Integrity Guaranteed**
Deleting a record in one table affects the related record in the other.

## SQL EXAMPLE

Create users table

```
CREATE TABLE users (
    id SERIAL PRIMARY KEY,
    username VARCHAR(50) NOT NULL,
    email VARCHAR(100) UNIQUE NOT NULL,
    created_at TIMESTAMP DEFAULT NOW()
);
```

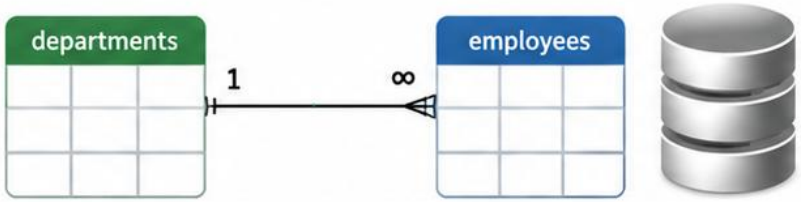
Create user_profiles table

```
CREATE TABLE user_profiles (
    user_id INTEGER PRIMARY KEY
        REFERENCES users(id)
        ON DELETE CASCADE,
    full_name VARCHAR(100) NOT NULL,
    phone VARCHAR(20),
    address VARCHAR(255),
    date_of_birth DATE
);
```

**KEY TAKEAWAY:** One-to-one relationships are ideal when data is logically separated but tightly linked, ensuring each record in one table has exactly one matching record in the other.

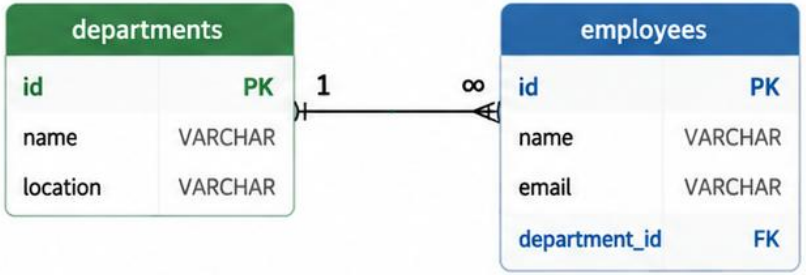
# One-to-Many

A single record in Table A relates to many records in Table B, but each record in Table B relates to only one record in Table A.




## EXAMPLE

One department can have many employees.
Each employee belongs to one department.




**employees.department_id** is a foreign key that references **departments.id.**
It ensures every employee belongs to an existing department.

## KEY CHARACTERISTICS

**One-to-many cardinality** (1:N)
One record in Table A → many records in Table B.

**Foreign Key on Many Side**
The "many" table contains a foreign key referencing the "one" table.

**Referential Integrity**
A department must exist before an employee can reference it.

**Supports Business Hierarchies**
Commonly used for master-detail relationships.

**Cascade Options**
ON DELETE CASCADE can be used to remove employees with a department.

## SQL EXAMPLE

**Create departments table**

```
CREATE TABLE departments (
   id SERIAL PRIMARY KEY,
   name VARCHAR(100) NOT NULL,
   location VARCHAR(100)
);
```

**Create employees table**

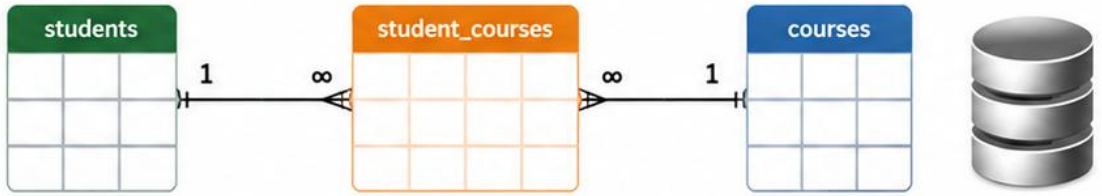
```
CREATE TABLE employees (
   id SERIAL PRIMARY KEY,
   name VARCHAR(100) NOT NULL,
   email VARCHAR(100) NOT NULL,
   department_id INTEGER NOT NULL,
   CONSTRAINT fk_employees_department
      FOREIGN KEY (department_id)
      REFERENCES departments(id)
      ON DELETE RESTRICT
);
```

**KEY TAKEAWAY:** Use one-to-many relationships to model hierarchical or master-detail data. They are the most common relationship type in relational databases.

One Department — 1 ∞ — Many Employees

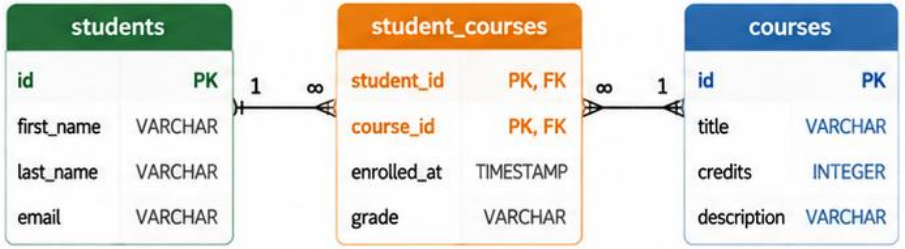
# Many-to-Many

Many records in Table A relate to many records in Table B,
and many records in Table B relate to many records in Table A.




## EXAMPLE

A student can enroll in many courses.
A course can have many students.




**student_courses** is a junction (bridge) table.

It contains foreign keys from both tables.

The combination of **student_id** and **course_id** is the primary key.

## KEY CHARACTERISTICS

**M:N — Many-to-many cardinality**
Many records in Table A relate to many records in Table B.

**Requires a Junction Table**
A third table stores the relationships using foreign keys from both tables.

**Composite Primary Key**
The junction table uses a composite primary key (student_id, course_id) to prevent duplicate relationships.

**Additional Attributes**
The junction table can store extra information about the relationship, such as enrollment date or grade.

**Supports Scalability**
Efficient and flexible for real-world relationships.

## SQL EXAMPLE

**Create students table**

```
CREATE TABLE students(
   id SERIAL PRIMARY KEY,
   first_name VARCHAR(50) NOT NULL,
   last_name VARCHAR(50) NOT NULL,
   email VARCHAR(100) UNIQUE
);
```

**Create courses table**

```
CREATE TABLE courses (
   id SERIAL PRIMARY KEY,
   title VARCHAR(100) NOT NULL,
   credits INTEGER NOT NULL,
   description TEXT
);
```

**Create junction table**

```
CREATE TABLE student_courses (
   student_id INTEGER NOT NULL REFERENCES students(id),
   course_id INTEGER NOT NULL REFERENCES courses(id),
   enrolled_at TIMESTAMP DEFAULT NOW(),
   grade VARCHAR(10),
   PRIMARY KEY (student_id, course_id)
);
```

**KEY TAKEAWAY:** Use many-to-many relationships when both sides can have multiple matches.
Model them using a junction table with foreign keys to both tables.

# NORMALIZATION OVERVIEW

*Normalization organizes data to reduce redundancy and eliminate update, insert, and delete anomalies.*

### Update Anomaly

The same fact repeated in many rows -- update one copy and the others silently go stale.

### Insert Anomaly

Cannot record one fact without also inventing unrelated, not-yet-known data.

### Delete Anomaly

Deleting one row accidentally destroys other, still-needed information.

### The Fix

Decompose overloaded tables into smaller, well-structured tables connected by foreign keys.

KEY TAKEAWAY  Normalization eliminates redundancy by giving every fact exactly one home -- most business schemas, including Lab 37's, target Third Normal Form (3NF).

# TRY IT YOURSELF — EXERCISE 3: MODEL THE CRM RELATIONSHIPS

*Reinforces: table relationships, the one-to-one, one-to-many, and many-to-many slides, and normalization as you just covered them.*

**STEPS (notes/lab37-er-notes.md)**

| Step | What To Do |
|---|---|
| **1 — Cardinalities** | State the cardinality between customer and account, in words. |
| **2 — Many-to-Many** | Name a CRM case needing a join table, and the columns that table holds. |
| **3 — Normalize** | Give one repeated value you would move into its own table, and why. |
| **4 — When Not To** | Name one case where duplicating a value is the right call. |

**CRM cardinalities**

```
customer 1 --- * account          (one customer, many accounts)
customer * --- * tag  via customer_tag(customer_id, tag_id)
denormalize deliberately: store the price as charged, not as looked up
```

TRY IT NOW — Complete Exercise 3 before continuing: exercises/exercise-03-relationship-model.md (workspace: examples/module-37-exercises).

# Normalization Overview

Normalization organizes data to reduce redundancy and improve data integrity.

**Denormalized**

| id | student | course | instructor |
|---|---|---|---|
| 1 | Alice | Java | Smith |
| 2 | Bob | Angular | Jones |
| 3 | Alice | Spring | Smith |

**Normalized**

| student_id | student |
|---|---|
| 1 | Alice |
| 2 | Bob |
| 3 | Alice |

| course_id | course | instructor |
|---|---|---|
| 101 | Java | Smith |
| 102 | Angular | Jones |
| 103 | Spring | Smith |

## What is Normalization?

Normalization is the process of structuring a relational database to reduce data redundancy and improve data integrity by organizing data into multiple related tables.

### Key Goals

- Eliminate duplicate data
- Maintain data consistency
- Simplify updates and maintenance
- Improve query performance

## The Normal Forms

**1NF — First Normal Form**
Eliminate repeating groups and ensure atomic values in each column.

**2NF — Second Normal Form**
Remove partial dependencies (all non-key attributes depend on the whole key).

**3NF — Third Normal Form**
Remove transitive dependencies (non-key attributes depend only on the key).

**BCNF — Boyce-Codd Normal Form**
Stronger version of 3NF (all determinants must be candidate keys).

**4NF — Fourth Normal Form**
Remove multi-valued dependencies.

**5NF — Fifth Normal Form**
Remove join dependencies.

## Example: From 1 Table to Normalized Tables

**Before Normalization (1NF)**

| student_id | student_name | course_id | course_name | instructor |
|---|---|---|---|---|
| 1 | Alice | 101 | Java | Smith |
| 1 | Alice | 102 | Angular | Jones |
| 2 | Bob | 103 | Spring | Smith |

**After Normalization (3NF)**

**Students**

| student_id | student_name |
|---|---|
| 1 | Alice |
| 2 | Bob |

**Courses**

| course_id | course_name | instructor |
|---|---|---|
| 101 | Java | Smith |
| 102 | Angular | Jones |
| 103 | Spring | Smith |

**Enrollments**

| student_id | course_id |
|---|---|
| 1 | 101 |
| 1 | 102 |
| 2 | 103 |

## Key Takeaways

- Normalization reduces redundancy and maintains data integrity.
- Each normal form removes specific types of dependencies.
- Proper normalization leads to cleaner design and better performance.

## Next Step

Design normalized tables for your domain model, starting with 1NF and progressing to 3NF.

# INDEX FUNDAMENTALS

*An index is a separate data structure that lets PostgreSQL find rows without scanning the whole table.*

### Book-Index Analogy

Like a book's index, it points straight to the right page instead of reading every page.

### Speeds Up Reads

Turns a full table scan into a fast lookup for WHERE, JOIN, and ORDER BY.

### Costs on Writes

Every INSERT, UPDATE, or DELETE must also maintain every index on that table.

### Index With Intent

Add an index because a real query pattern needs it -- not on every column by default.

KEY TAKEAWAY  An index trades write cost and storage for dramatically faster reads -- Module 38 goes deep on this trade-off, but the core idea starts here.

37 **MODULE 37:** PostgreSQL Database Fundamentals

# Index Fundamentals

Indexes improve the speed of data retrieval operations on a database table.
They work like the index in a book—helping the database find rows quickly without scanning the entire table.

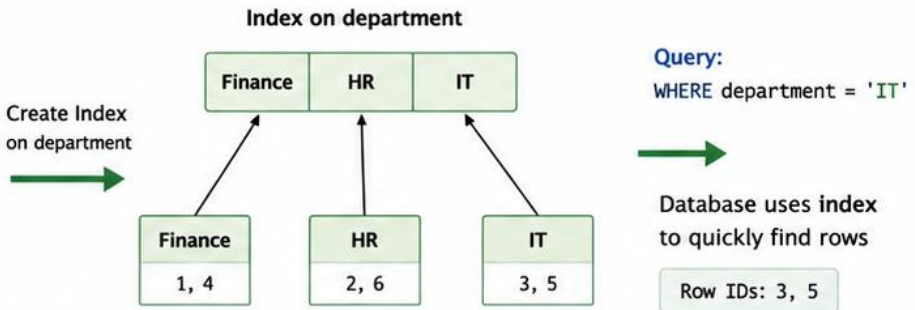
### Goal of Indexes

Reduce disk I/O, speed up searches and joins, and improve overall query performance.

## How an Index Works (Example)

**Table: employees**

| id | name | department | salary |
|---|---|---|---|
| 1 | Alice | Finance | 70000 |
| 2 | Bob | HR | 60000 |
| 3 | Carol | IT | 80000 |
| 4 | David | Finance | 65000 |
| 5 | Eve | IT | 90000 |
| 6 | Frank | HR | 55000 |




## Key Points

**Speeds Up Searches**
Allows the database to locate rows without scanning the entire table.

**Extra Storage**
Indexes consume additional disk space and memory.

**Slower Writes**
INSERT, UPDATE, DELETE operations are slower because indexes must be maintained.

**Balance is Key**
Create indexes on columns used in WHERE, JOIN, ORDER BY, GROUP BY, but avoid over-indexing.

## Common Index Types in PostgreSQL

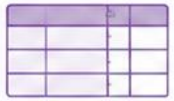
**B-Tree Index**

Default index type.
Best for equality and range queries (<, <=, >, >=, BETWEEN).

**Unique Index**

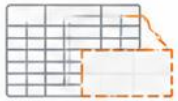
Ensures all values in the indexed column(s) are unique.

**Composite Index**



Index on multiple columns.
Column order matters for query optimization.

**Partial Index**



Indexes only a subset of rows that meet a condition.
Smaller and more efficient.

**Expression Index**


Indexes the result of an expression or function.
Useful for computed values.

## When to Create an Index

- Columns used in WHERE clauses
- Columns used in JOIN conditions
- Columns used in ORDER BY / GROUP BY
- Columns used in DISTINCT
- Columns with high selectivity (few unique values)
- Avoid columns with very low selectivity (e.g., Boolean, status flags)

**Remember:** Indexes are powerful but come with a cost.
Create the right indexes, not too many, and review them regularly.

**PostgreSQL Tip**
- Use EXPLAIN (ANALYZE, BUFFERS) to verify index usage.
- Drop unused indexes to save space and improve write performance.

# B-TREE INDEXES

*B-Tree is PostgreSQL's default index type -- efficient for equality, range, and sorted queries.*

```
CREATE INDEX idx_customer_status ON customer (status);
-- USING btree is the default and can be omitted
```

Works for =, <, <=, >, >=, BETWEEN, and IN comparisons.
Supports ORDER BY directly -- the index is already sorted.
The right default for most foreign keys and frequently filtered columns.

KEY TAKEAWAY  CREATE INDEX defaults to a B-Tree, PostgreSQL's general-purpose index -- it is almost always the right first choice unless a query pattern calls for something more specialized.

37 **MODULE 37:** PostgreSQL Database Fundamentals

# B-Tree Indexes

B-Tree is the default index type in PostgreSQL. It is a balanced tree data structure that keeps index entries sorted, allowing efficient search, insert, update, and delete.

### What is a B-Tree Index?

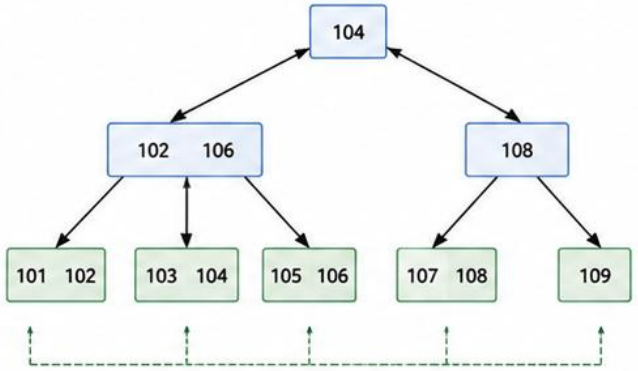
A B-Tree (Balanced Tree) index stores data in a sorted tree structure.
Each node can have multiple keys and multiple children.
All leaf nodes are at the same level, keeping the tree balanced for consistent performance.

## How a B-Tree Index Works (Example)

**Example Table: employees**

| id (PK) | name | department | salary |
|---|---|---|---|
| 101 | Alice | Finance | 70000 |
| 102 | Bob | HR | 60000 |
| 103 | Carol | IT | 80000 |
| 104 | David | Finance | 65000 |
| 105 | Eve | IT | 90000 |
| 106 | Frank | HR | 55000 |
| 107 | Grace | Finance | 75000 |
| 108 | Hannah | IT | 68000 |
| 109 | Ivy | HR | 58000 |

**B-Tree Index on id**




All leaf nodes are at the same level

**Search Example:**
Find id = 105

1. Start at root node [104]
   105 > 104, go right
2. At node [108]
   105 < 108, go left
3. At node [102, 106]
   105 is between 102 and 106, go to middle child
4. Reached leaf [105, 106]
   **105 found!**

## Key Characteristics

**Balanced Tree Structure**
The tree remains balanced after inserts and deletes, ensuring consistent performance.

**Ordered Data**
Index entries are stored in sorted order, enabling fast searches and range queries.

**Efficient Operations**
Optimized for equality searches and range-based queries.

**Default Index Type**
B-Tree is the default index type for most column data types in PostgreSQL.

### Good For

- Equality comparisons (=)
- Range queries (>, >=, <, <=, BETWEEN)
- ORDER BY and GROUP BY operations
- JOIN operations

### Not Ideal For

- Full-text search
- Leading wildcard searches (LIKE '%abc')
- Very low-selectivity columns (e.g., gender, status flags)

### Advantages

- Fast search performance
- Handles large datasets efficiently
- Maintains performance as data grows
- Well-suited for most transactional workloads

### Example in PostgreSQL

```
-- Create a B-Tree index on salary column
CREATE INDEX idx_employees_salary
  ON employees (salary);
```

**Key Takeaway:** B-Tree indexes provide a balanced, sorted structure that accelerates data retrieval. They are the most commonly used indexes in relational databases and a foundation for query performance.

**Tip:** Monitor index usage with EXPLAIN (ANALYZE, BUFFERS) and remove unused indexes to save storage and improve write performance.

# UNIQUE INDEXES

*A UNIQUE index enforces distinct values while also speeding up lookups on that column.*

```
CREATE UNIQUE INDEX idx_customer_email ON customer (email);
```

Behaves exactly like a UNIQUE constraint -- in fact, UNIQUE constraints are implemented as unique indexes internally.
A duplicate insert is rejected with the same 'violates unique constraint' error.
PRIMARY KEY and UNIQUE columns already get a unique index automatically -- no separate CREATE UNIQUE INDEX is needed for those.

KEY TAKEAWAY  PostgreSQL implements every UNIQUE constraint, and every PRIMARY KEY, as a unique index under the hood -- CREATE UNIQUE INDEX is the same mechanism used explicitly.

37 MODULE 37: PostgreSQL Database Fundamentals

# Unique Indexes

A **Unique Index** enforces uniqueness of values in one or more columns.
It ensures that no two rows in a table can have the same value(s) in the indexed column(s).

## Why Use Unique Indexes?

- Enforce business rules and data integrity
- Prevent duplicate data
- Improve query performance for equality searches
- Often used to support PRIMARY KEY and UNIQUE constraints

## Example

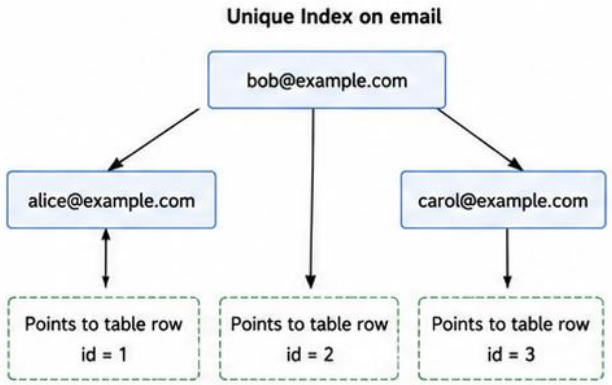
**Table: users**

| id (PK) | username | email | role |
|---|---|---|---|
| 1 | alice | alice@example.com | ADMIN |
| 2 | bob | bob@example.com | USER |
| 3 | carol | carol@example.com | USER |

**Unique Index on email**

```
CREATE UNIQUE INDEX ux_users_email
  ON users (email);
```

## How It Works




The index stores **unique** email values in sorted order.

Each value points to the corresponding row in the table.

## Key Characteristics

**Enforces Uniqueness**
Prevents duplicate values in the indexed column(s).

**Allows NULL Values**
PostgreSQL allows multiple NULLs in a unique index (NULLs are not considered equal).

**Improves Performance**
Speeds up equality lookups and joins on the indexed column(s).

**Can Be Composite**
Unique indexes can be created on multiple columns to enforce combined uniqueness.

## Enforcement Example

**Valid Insert**

```
INSERT INTO users (username, email, role)
VALUES ('dave', 'dave@example.com', 'USER');
```

**Invalid Insert (Duplicate Email)**

```
INSERT INTO users (username, email, role)
VALUES ('david', 'alice@example.com', 'USER');
```

**Error Returned**

ERROR: duplicate key value violates unique constraint "ux_users_email"

DETAIL: Key (email)=(alice@example.com) already exists.

## Unique Index vs Other Indexes

| Index Type | Allows Duplicates? | Enforces Uniqueness? | Typical Use Case |
|---|---|---|---|
| B-Tree Index | Yes | No | General search and performance |
| Unique Index | No | Yes | Enforce uniqueness, integrity |
| Primary Key | No | Yes | Entity identity (not null + unique) |

## Best Practices

- Create unique indexes to enforce business rules.
- Use meaningful column(s) with high selectivity.
- Avoid unnecessary unique indexes (write overhead).
- Use composite unique indexes for multi-column rules.
- Monitor index usage with EXPLAIN and pg_stat_user_indexes.

**Key Takeaway:** Unique indexes are essential for maintaining data integrity by ensuring that values in one or more columns remain unique. They also improve query performance for equality lookups while enforcing important business rules.

**Tip:** Use UNIQUE constraints or unique indexes based on whether the rule is part of the schema definition or created later for performance.

# COMPOSITE INDEXES

*A composite index spans two or more columns, sorted together in the order they're declared.*

```
CREATE INDEX idx_account_customer_status ON account (customer_id, status);
```

Speeds up queries that filter on the leading column, or the leading column plus more.
Column order matters -- (customer_id, status) is not the same index as (status, customer_id).
Module 38 covers choosing composite column order in depth, using real query patterns.

KEY TAKEAWAY  A composite index sorts by its first column, then its second, and so on -- get the column order right and one index can serve several related queries.

# TRY IT YOURSELF — EXERCISE 4: PLAN INDEXES FOR REAL QUERIES

*Reinforces: index fundamentals, B-tree, unique, and composite indexes as you just covered them, tied to real CRM queries.*

**STEPS (notes/lab37-indexes.md)**

| Step | What To Do |
|---|---|
| **1 — Query First** | Write the two queries the CRM runs most, before naming any index. |
| **2 — Index Per Query** | Give the index that serves each, including column order for composites. |
| **3 — Column Order** | Say why the composite's column order matters. |
| **4 — Cost** | Say what every extra index costs on write. |

**Index plan**

```
q1 find by email          -> unique index on customer(email)
q2 accounts for a customer, newest first
   -> index on account(customer_id, opened_at desc)
```

TRY IT NOW — Complete Exercise 4 before continuing: exercises/exercise-04-index-plan.md (workspace: examples/module-37-exercises).

# Composite Indexes

A composite index is an index on **two or more columns**. It improves query performance when filtering or sorting by multiple columns.

**Example Table: orders**

| order_id (PK) | customer_id | order_date | status | total_amount |
|---|---|---|---|---|
| 1001 | 501 | 2025-05-01 | NEW | 125.00 |
| 1002 | 501 | 2025-05-02 | SHIPPED | 220.00 |
| 1003 | 502 | 2025-05-01 | NEW | 75.00 |
| 1004 | 503 | 2025-05-03 | DELIVERED | 320.00 |
| 1005 | 501 | 2025-05-03 | CANCELLED | 60.00 |

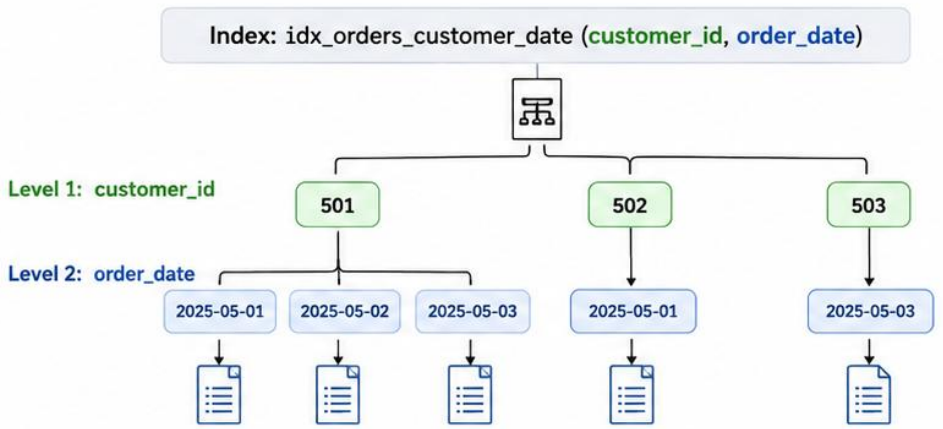
```
CREATE INDEX idx_orders_customer_date
  ON orders (customer_id, order_date);
```

This composite index is ordered as:

customer_id → order_date

The column order matters. PostgreSQL can use the index from **left to right**.

## How a Composite Index Works




Efficient for queries filtering by **customer_id** alone, or by **customer_id** and **order_date** together.

## Best Practices

- Place the **most selective** column first.
- Match the **index column order** with your most common query patterns.
- Avoid unnecessary composite indexes; they increase **write cost**.
- Check index usage with **EXPLAIN ANALYZE**.

**Rule of Thumb:** Use composite indexes when queries filter by multiple columns together.

PostgreSQL

# POSTGRESQL SEQUENCES

*A sequence is a standalone database object that generates a strictly increasing series of numbers.*

```
CREATE SEQUENCE account_number_seq START 100000 INCREMENT 1;

SELECT nextval('account_number_seq');  -- 100000
SELECT nextval('account_number_seq');  -- 100001
```

Sequences are safe under concurrency -- two simultaneous callers never receive the same value.
A gap can appear if a transaction calling nextval() rolls back -- this is normal, not a bug.
GENERATED ... AS IDENTITY, covered next, manages a sequence for you automatically.

KEY TAKEAWAY  A sequence generates guaranteed-unique, strictly increasing numbers, safely even under heavy concurrent access -- it's the engine behind every identity-column primary key.

# PostgreSQL Sequences

A sequence is a database object that generates a unique, sequential number that can be used for primary keys or other unique identifiers.

## Why Use Sequences?

- Generate unique values automatically
- Ensure no duplicate keys
- Simplify primary key generation
- Works well with high-concurrency applications

## How Sequences Work

1. **Create Sequence** — Define a sequence with a start value, increment, etc.
2. **Get Next Value** — Use nextval() to retrieve the next number in the sequence.
3. **Use in Table** — Insert the value into your table (e.g., as a primary key).
4. **Sequence Increments** — The sequence automatically moves to the next value.

## Creating and Using a Sequence

```
CREATE SEQUENCE employee_id_seq
     START 1
     INCREMENT 1
     MINVALUE 1
     CACHE 10;

CREATE TABLE employees (
    id BIGINT PRIMARY KEY DEFAULT nextval('employee_id_seq'),
    name VARCHAR(100),
    email VARCHAR(100)
);
```

## Common Sequence Operations

| Operation | Example |
|---|---|
| Get next value | `SELECT nextval('employee_id_seq');` |
| Get current value | `SELECT currval('employee_id_seq');` |
| View sequence value | `SELECT last_value FROM employee_id_seq;` |
| Reset sequence | `ALTER SEQUENCE employee_id_seq RESTART WITH 1;` |

## Example Output

| Operation | Result |
|---|---|
| nextval() | 1 |
| nextval() | 2 |
| nextval() | 3 |
| currval() | 3 |
| last_value | 3 |

## Best Practices

- Use sequences for primary key generation.
- Set an appropriate increment value (typically 1).
- Use CACHE for better performance in high-load systems.
- Avoid manually updating sequence values.
- Monitor sequence usage in production.

**Key Takeaway** — PostgreSQL sequences provide a simple and reliable way to generate unique, sequential numbers, making them ideal for primary keys and other unique identifiers in your database schema.

# IDENTITY COLUMNS

*GENERATED AS IDENTITY is PostgreSQL's modern, SQL-standard way to auto-generate primary key values.*

| Form | Behavior |
| --- | --- |
| **GENERATED ALWAYS AS IDENTITY** | PostgreSQL always supplies the value -- an explicit INSERT value is rejected unless OVERRIDING SYSTEM VALUE is used. |
| **GENERATED BY DEFAULT AS IDENTITY** | PostgreSQL supplies the value only if the INSERT omits it -- an explicit value is allowed. |

KEY TAKEAWAY  GENERATED ALWAYS AS IDENTITY is the modern, standards-compliant default for PostgreSQL primary keys -- it manages a backing sequence for you automatically.

# Identity Columns

Identity columns (**SQL standard**) automatically generate **unique values** for a column, typically used for primary keys. They are easier to use and manage than sequences + default.

## How Identity Columns Work


**Insert Request**
No value provided for identity column

**Database**
Automatically generates next value

**Value Assigned**
Unique value is assigned

**Row Stored**
Row inserted with generated value

## Create Table with Identity Column

```
CREATE TABLE employees (
  employee_id BIGINT GENERATED ALWAYS AS IDENTITY
  PRIMARY KEY,
  first_name VARCHAR(50) NOT NULL,
  last_name VARCHAR(50) NOT NULL,
  email VARCHAR(100) UNIQUE,
  created_at TIMESTAMP DEFAULT now()
);
```

**GENERATED ALWAYS**
Database always generates the value

**AS IDENTITY**
Marks column as an identity column

## Insert Without Providing ID

```
INSERT INTO employees (first_name, last_name, email)
VALUES
('John', 'Smith', 'john.smith@example.com');

INSERT INTO employees (first_name, last_name, email)
VALUES
('Sara', 'Johnson', 'sara.johnson@example.com');

INSERT INTO employees (first_name, last_name, email)
VALUES
('Michael', 'Brown', 'michael.brown@example.com');
```

### Generated Values

| employee_id (PK) | first_name | last_name | email |
|---|---|---|---|
| 101 | John | Smith | john.smith@example.com |
| 102 | Sara | Johnson | sara.johnson@example.com |
| 103 | Michael | Brown | michael.brown@example.com |
| ... | ... | ... | ... |

## Identity vs Sequence + Default

| Identity Column | Sequence + Default |
|---|---|
| SQL standard (since PostgreSQL 10) | Requires creating sequence separately |
| Simpler syntax and easier to use | More manual steps |
| Automatically manages the underlying sequence | Default `nextval('sequence_name')` |
| Easier to alter (ADD/DROP IDENTITY) | More flexible in some advanced scenarios |
| Preferred approach for new development | Useful when sharing sequence across tables |

## Key Points

- Identity columns guarantee **unique, ever-increasing** values.
- Use `GENERATED ALWAYS` for values fully controlled by the database.
- Use `GENERATED BY DEFAULT` to allow explicit value insertion when needed.
- Identity columns are the **recommended** way to define auto-incrementing primary keys.

**Best Practice:** Use identity columns for primary keys unless you have a specific requirement that needs sequence sharing or advanced control.

PostgreSQL

# CONNECTING TO POSTGRESQL WITH PSQL

*psql is PostgreSQL's official command-line client -- the fastest way to run SQL directly against the server.*

```
psql -h localhost -p 5432 -U crm_app -d crmdb
```

| Meta-Command | Purpose |
|---|---|
| **\l** | List all databases on the server. |
| **\c dbname** | Connect to a different database. |
| **\dt** | List tables in the current schema's search path. |
| **\d tablename** | Describe a table -- columns, types, and constraints. |
| **\q** | Quit psql. |

KEY TAKEAWAY  psql is the fastest way to verify a schema exists and behaves as designed -- \dt and \d are the two meta-commands students will use most in the lab.

# Connecting to PostgreSQL with `psql`

`psql` is the PostgreSQL command-line interface. It allows you to connect to a database server, run SQL queries, manage schemas, and perform administrative tasks.

## 1. Basic Connection Syntax

```
psql -h <host> -p <port> -U <username> -d <database>
```

- **-h <host>** — Database server host (default: localhost)
- **-p <port>** — Port number (default: 5432)
- **-U <username>** — Database user
- **-d <database>** — Target database to connect to

## 2. Common Examples

```
psql -U postgres                                  # Connect to default 'postgres' database
psql -U appuser -d empdb                          # Connect to empdb as appuser
psql -h db.example.com -p 5432 -U appuser -d empdb  # Remote connection
```

## psql

```
$ psql -h localhost -p 5432 -U postgres -d empdb
Password for user postgres:
psql (16.2)
Type "help" for help.

empdb=# \conninfo
You are connected to database "empdb" as user "postgres"
on host "localhost" (address "127.0.0.1") at port "5432".

empdb=# \dt
             List of relations
 Schema |      Name      | Type  |  Owner
--------+----------------+-------+----------
 public | employee       | table | postgres
 public | department     | table | postgres
 public | job_role       | table | postgres
(3 rows)

empdb=#
```

## Useful psql Commands

| | |
|---|---|
| `\l` | List all databases |
| `\c <db>` | Connect to a database |
| `\dt` | List tables in current schema |
| `\d <table>` | Describe table structure |
| `\du` | List database users |
| `\conninfo` | Show connection info |
| `\q` | Quit psql |

**Tip:** Type `\\?` to see all psql meta-commands.

**Pro Tip** If you omit `-h`, `-p`, and `-d`, psql uses defaults: Host: localhost | Port: 5432 | Database: postgres

# PGADMIN OVERVIEW

*pgAdmin is PostgreSQL's official graphical administration tool -- browse schemas, run queries, and inspect data visually.*

### Object Browser

Navigate servers, databases, schemas, and tables in a familiar tree view.

### Query Tool

Write and run SQL with syntax highlighting and a results grid.

### Table Inspector

View columns, constraints, and indexes without writing a single query.

### EXPLAIN Visualizer

Renders a query's execution plan as a graphical diagram -- a Module 38 preview.

KEY TAKEAWAY  pgAdmin gives a visual view of everything psql shows at the command line -- most PostgreSQL developers use both, switching based on the task.

# pgAdmin Overview

**pgAdmin** is the official, open-source administration and development platform for PostgreSQL.
It provides a powerful GUI to manage databases, objects, queries, users, permissions, and much more.

## Key Capabilities

**Database Management**
Create, modify, and manage databases, schemas, tables, and other objects.

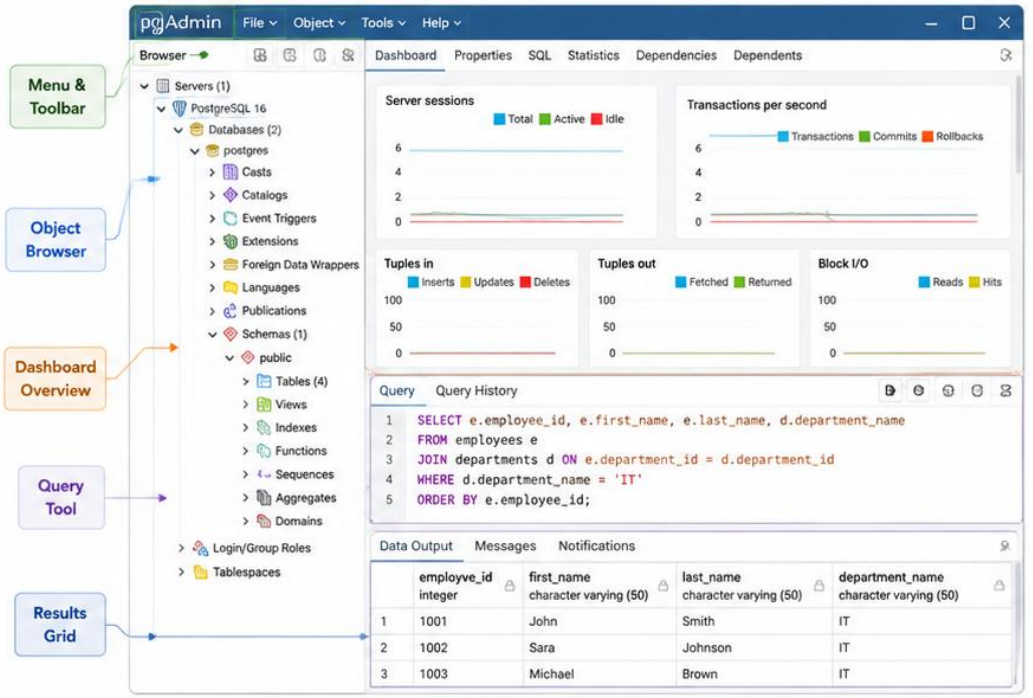
**Query Tool**
Write, run, and optimize SQL queries with syntax highlighting and results grid.

**Visual Tools**
ERD tool, query history, explain plans, and performance monitoring.

**Security & Access**
Manage roles, privileges, and object-level permissions.

**Backup & Restore**
Easily back up and restore databases and objects.

## pgAdmin Interface




## Common Tasks in pgAdmin

**Create Database**
Wizard-based database creation.

**Run SQL Queries**
Use the Query Tool to execute and analyze SQL.

**Browse & Edit Data**
View and edit table data directly.

**Manage Users & Roles**
Create roles and set permissions.

**Backup / Restore**
Export or restore databases with ease.

**Monitor Performance**
View activity, locks, and query statistics.

**Tip:** pgAdmin works on Windows, macOS, and Linux and connects to any PostgreSQL database.

## Why pgAdmin?

- Official PostgreSQL tool
- Feature-rich and extensible
- Ideal for DBAs, developers, and analysts

pgAdmin 4

# CONNECTING JAVA TO POSTGRESQL

*A Java application talks to PostgreSQL through JDBC -- the same driver-based pattern used across every relational database.*

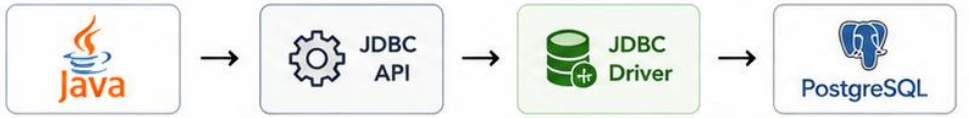
```
String url = "jdbc:postgresql://localhost:5432/crmdb";
String user = "crm_app";
String pass = System.getenv("CRM_DB_PASSWORD");

try (Connection conn = DriverManager.getConnection(url, user, pass)) {
    // conn is a live PostgreSQL session
}
```

KEY TAKEAWAY  Every JDBC connection needs three things: a jdbc:postgresql:// URL, credentials, and the PostgreSQL JDBC driver on the classpath -- Spring Boot automates all three from configuration.

# Connecting Java to PostgreSQL

Java applications connect to PostgreSQL using JDBC (Java Database Connectivity). The PostgreSQL JDBC Driver provides the implementation for communicating between Java and the PostgreSQL database.

### Connection Flow




**Java Application**
Your Java program

**JDBC API**
Standard interface for database access

**PostgreSQL JDBC Driver**
Implements the JDBC API for PostgreSQL

**PostgreSQL Database**
Stores and manages your data

### 3. Example: Connecting to PostgreSQL in Java

```
import java.sql.Connection;
import java.sql.DriverManager;
import java.sql.SQLExqection;

public class PostgresConnectionExample {
   public static void main(String[] args) {
       String url      = "jdbc:postgresql://localhost:5432/employeedb";
       String user     = "dbuser";
       String password = "securePassword";

        try (Connection conn = DriverManager.getConnection(url, user, password)) {
            System.out.println("✅ Connected to PostgreSQL successfully!");
       } catch (SQLEcqeation e) {
            System.err.println("❌ Connection failed: " + e.getMessage());
       }
   }
}
```

### 1. Add PostgreSQL JDBC Driver Dependency (Maven)

```
<dependency>
    <groupId>org.postgresql</groupId>
    <artifactId>postgresql</artifactId>
    <version>42.7.3</version>
</dependency>
```

### 2. JDDC Connection URL Structure

```
jdbc:postgresql://<host>:<port>/<database>?<parameters>
```

- `jdbc:postgresql` : JDBC protocol for PostgreSQL
- `host` : Database server hostname or IP
- `port` : Database port (default: 5432)
- `database` : Target database name
- `parameters` : Optional connection parameters

### 4. Common Connection Parameters

| Parameter | Description | Example |
|---|---|---|
| ssl | Enable SSL connection | ?ssl=true |
| connectTimeout | Connection timeout in seconds | ?connectTimeout=10 |
| currentSchema | Default schema to use | ?currentSchema=public |
| ApplicationName | Name of the application | ?ApplicationName=MyApp |

### Best Practices

- Use connection pooling (e.g., HikariCP) in production applications.
- Store database credentials in environment variables or configuration files.
- Always close connections, statements, and result sets.

**Tip:** Use try-with-resources to automatically close connections and prevent resource leaks.

PostgreSQL

# POSTGRESQL JDBC DRIVER

*The PostgreSQL JDBC driver is the library that translates Java's JDBC API calls into the PostgreSQL wire protocol.*

```
<!-- Maven -->
<dependency>
  <groupId>org.postgresql</groupId>
  <artifactId>postgresql</artifactId>
  <scope>runtime</scope>
</dependency>
```

Registers itself with java.sql.DriverManager automatically -- no manual Class.forName() needed.
scope=runtime is intentional -- application code depends on the JDBC API, not this driver directly.
Spring Boot resolves the exact compatible version through its dependency management.

KEY TAKEAWAY  Adding the org.postgresql:postgresql dependency is the one setup step every PostgreSQL Java project needs -- everything else in the connection flow is standard JDBC.

# PostgreSQL JDBC Driver

The **PostgreSQL JDBC Driver** enables Java applications to connect to and interact with PostgreSQL databases.

## Role in Java Applications


**Java Application**
Your Java program

**JDBC API**
Standard interface for database access

**JDBC Driver**
Implements JDBC API for PostgreSQL

**PostgreSQL Database**
Stores and manages your data

## Add Dependency (Maven)

```xml
<dependency>
    <groupId>org.postgresql</groupId>
    <artifactId>postgresql</artifactId>
    <version>42.7.3</version>
</dependency>
```

## Add Dependency (Gradle)

```
implementation 'org.postgresql:postgresql:42.7.3'
```

## Example: Load the Driver and Get Connection

```java
import java.sql.Connection;
import java.sql.DriverManager;
import java.sql.SQLEcqpaction;

public class PostgreSQLConnectionExample {
    public static void main(String[] args) {
        String url = "jdbc:postgresql://localhost:5432/employeedb";
        String user = "dbuser";
        String password = "securePassword";

        try (Connection conn = DriverManager.getConnection(url, user, password)) {
            System.out.println("🌿Connected to PostgreSQL successfully!");
        } catch (SQLEcqpation e) {
            System.err.println("❌ Connection failed: " + e.getMessage());
    }  }
}
```

## JDBC Driver Types

| Driver Type | Description | PostgreSQL Support |
|---|---|---|
| Type 1 | JDBC-ODBC Bridge Driver | Not supported |
| Type 2 | Native-API (partially Java) | Not supported |
| Type 3 | Network Protocol Driver | Not supported |
| Type 4 | Pure Java Driver (Thin Driver) | ✅ Supported (Recommended) |

## Why Type 4 (Pure Java) Driver?

- ✅ 100% Java – platform independent
- ✅ High performance and lightweight
- ✅ No native libraries required
- ✅ Fully supports PostgreSQL features
- ✅ Recommended and actively maintained

## Common Driver Class

For PostgreSQL Type 4 driver, the class is automatically registered via SPI.

```
org.postgresql.Driver
```

## Driver JAR

The PostgreSQL JDBC Driver is packaged as a JAR file.


## Best Practices

- Always use the latest stable driver version.
- Store the JAR in your local repository or dependency manager.
- Use **try-with-resources** to ensure connections are closed.
- Do not hardcode credentials; use configuration files or environment variables.

**Tip:** Use the official PostgreSQL JDBC Driver for maximum compatibility and performance.

PostgreSQL

# CONNECTION URL STRUCTURE

*Every PostgreSQL JDBC URL follows the same four-part shape: protocol, host, port, and database.*

| Part | Example | Meaning |
|---|---|---|
| **Protocol** | jdbc:postgresql:// | Tells DriverManager which driver should handle this URL. |
| **Host** | localhost | Where the PostgreSQL server is running. |
| **Port** | 5432 | PostgreSQL's conventional port -- can be omitted if using the default. |
| **Database** | /crmdb | Which database on that server to connect to. |

KEY TAKEAWAY  jdbc:postgresql://host:port/database -- get this exact shape memorized; Module 39's application.properties datasource URL uses this identical format.

# Connection URL Structure

A JDBC connection URL tells the PostgreSQL JDBC Driver how to connect to your database.

## General Format

```
jdbc:postgresql://host:port/database??param1=value1&param2=value2
```

| Protocol | Host | Port | Database | Parameters |
|---|---|---|---|---|
| Always jdbc:postgresql | Database server hostname or IP | Database server port (default: 5432) | Target database name | Optional connection parameters |

## Example Connection URLs

| Description | JDBC Connection URL |
|---|---|
| Local default connection | `jdbc:postgresql://localhost:5432/mydb` |
| Local with custom port | `jdbc:postgresql://localhost:6543/mydb` |
| Remote host | `jdbc:postgresql://dbserver.example.com:5432/mydb` |
| With parameters | `jdbc:postgresql://localhost:5432/mydb?ssl=true&connectTimeout=10&ApplicationNaame=MyApp` |
| Using IP address | `jdbc:postgresql://192.168.1.50:5432/mydb` |

## Common Connection Parameters

| Parameter | Description | Example |
|---|---|---|
| ssl | Enable SSL encryption | ?ssl=true |
| connectTimeout | Connection timeout in seconds | ?connectTimeout=10 |
| socketTimeout | Socket read timeout in seconds | ?socketTimeout=30 |
| ApplicationName | Name of the application | ?ApplicationName=MyApp |
| currentSchema | Default schema to use | ?currentSchema=public |
| loginTimeout | Login timeout in seconds | ?loginTimeout=5 |

## Example in Java

```
String url = "jdbc:postgresql://localhost:5432/employeedb?" +
             "ssl=true&connectTimeout=10&ApplicationName=EmployeeApp";
String user = "dbuser";
String password = "securePassword";

Connection conn = DriverManager.getConnection(url, user, password);
```

## Where Each Part Comes From

**Host & Port**
Provided by your DBA or database administrator.

**Database**
The name of the database you want to connect to.

**Parameters**
Optional settings to control connection behavior.

## Best Practices

- Use environment variables or configuration files for connection URLs.
- Always enable SSL (ssl=true) in production environments.
- Set appropriate timeouts to avoid hanging connections.
- Use ApplicationName to identify your application in PostgreSQL logs.

## Quick Tip

If no port is specified, the driver uses the default PostgreSQL port 5432.
Keep your credentials secure—never hardcode them in source code.

PostgreSQL

# DATABASE USERS AND ROLES

*PostgreSQL has a single unified concept, the role, that represents both a user and a group.*

```
CREATE ROLE crm_app WITH LOGIN PASSWORD 'change_me' NOSUPERUSER;

-- roles can be grouped
CREATE ROLE crm_readonly;
GRANT crm_readonly TO crm_app;
```

LOGIN makes a role usable for connecting -- roles without LOGIN act as pure groups.
A superuser role bypasses every permission check -- never used by an application.
\du in psql lists every role on the server.

KEY TAKEAWAY  PostgreSQL calls both individual users and groups 'roles' -- CREATE ROLE ... WITH LOGIN is how an application, not a human, gets its own database identity.

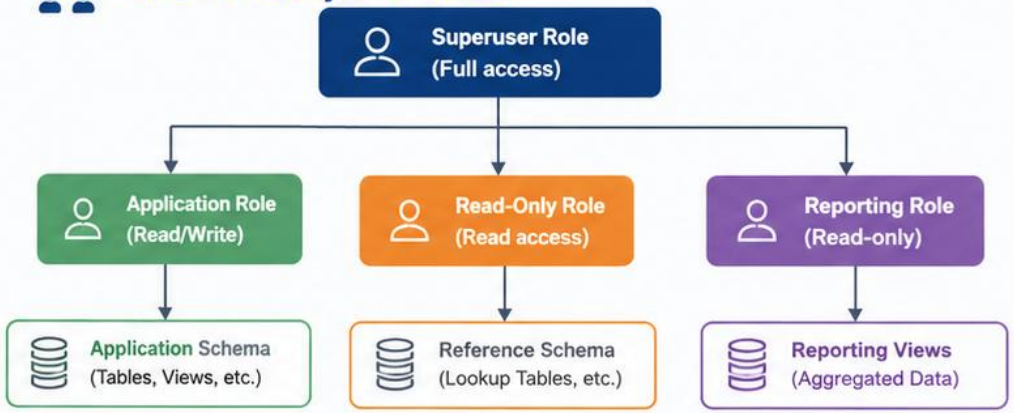
# Database Users and Roles

Control access to PostgreSQL resources using roles. Users are roles that can log in.

## Key Concepts

- **Role:** A named entity that can own objects and have privileges.
- **User:** A role that can authenticate (has LOGIN privilege).
- **Privileges:** Permissions to perform actions on database objects.
- **Ownership:** Object owners automatically have all privileges.
- **Principle of Least Privilege:** Grant only the permissions needed.

## Role Hierarchy and Access




## Common Privileges

| Privilege | Description |
|---|---|
| CONNECT | Allows connecting to a database |
| USAGE | Allows using a schema |
| SELECT | Allows reading data from tables/views |
| INSERT | Allows inserting data |
| UPDATE | Allows updating data |
| DELETE | Allows deleting data |
| ALL PRIVILEGES | Grants all privileges on an object |

## Example SQL

### Create Roles

```
CREATE ROLE app_user LOGIN PASSWORD 'App@123';
CREATE ROLE readonly_user LOGIN PASSWORD 'Read@123';
CREATE ROLE reporting_user LOGIN PASSWORD 'Report@123';
CREATE ROLE app_role;

GRANT app_role TO app_user;
```

### Grant Privileges

```
GRANT CONNECT ON DATABASE employee_db TO app_role;
GRANT USAGE ON SCHEMA public TO app_role;
GRANT SELECT, INSERT, UPDATE, DELETE
   ON ALL TABLES IN SCHEMA public TO app_role;
GRANT SELECT ON ALL TABLES IN SCHEMA public
   TO readonly_user, reporting_user;
```

## Best Practices

**Follow Least Privilege**
Grant only necessary permissions.

**Use Roles for Groups**
Assign users to roles instead of granting privileges directly.

**Avoid Superuser for Applications**
Use dedicated roles.

**Review Privileges Regularly**
Audit access and remove unnecessary permissions.

**Separate Duties**
Use different roles for development, reporting, and administration.

# PERMISSIONS AND LEAST PRIVILEGE

*Grant a role exactly the permissions it needs to do its job, and nothing more.*

```
GRANT CONNECT ON DATABASE crmdb TO crm_app;
GRANT USAGE ON SCHEMA public TO crm_app;
GRANT SELECT, INSERT, UPDATE ON customer, account TO crm_app;
GRANT USAGE ON ALL SEQUENCES IN SCHEMA public TO crm_app;
```

KEY TAKEAWAY  An application role gets exactly the GRANTs its code path needs -- CONNECT, USAGE, and table-level SELECT/INSERT/UPDATE, never superuser or DROP/DELETE it doesn't use.

# TRY IT YOURSELF — EXERCISE 5: PLAN THE ROLE AND CONNECTION

*Reinforces: psql, the JDBC driver, connection URL structure, and the users, roles, and least-privilege slides you just covered.*

## STEPS (notes/lab37-role-connection.md)

| Step | What To Do |
|---|---|
| **1 — JDBC URL** | Write the connection URL shape for the CRM database, with no password in it. |
| **2 — App Role** | Say what crm_app may do and what it must not. |
| **3 — Least Privilege** | Give one thing the app role is denied and the reason. |
| **4 — Secret Handling** | Say where the password lives, given it is never in Git. |

## Role and connection

```
jdbc:postgresql://localhost:5432/crm   user=crm_app  (password injected)
grant select, insert, update, delete on crm tables to crm_app
no ddl, no superuser, no drop -- migrations run as a different role
```

TRY IT NOW — Complete Exercise 5 before continuing: exercises/exercise-05-role-and-connection.md (workspace: examples/module-37-exercises).

# Permissions and Least Privilege

Grant only the permissions required. Nothing more.

## What Are Permissions?

**Permissions (privileges)** control what actions a user or role can perform on database objects.

## Permission Levels

- **Object Level**
  Apply to specific objects (tables, views, sequences, functions).
- **Schema Level**
  Apply to all objects within a schema.
- **Database Level**
  Apply to the entire database.
- **System Level**
  Apply to database-wide administrative actions.

## Common Object Privileges

| Privilege | Description |
|---|---|
| SELECT | Read data from tables or views |
| INSERT | Insert new rows |
| UPDATE | Modify existing rows |
| DELETE | Remove rows |
| REFERENCES | Create foreign keys |
| EXECUTE | Execute functions or procedures |
| USAGE | Use schemas, sequences, types |

## Example: Applying Least Privilege

**Application role** needs to read and write employee data, but should not be able to drop or alter objects.

```
CREATE ROLE app_user LOGIN PASSWORD 'App@123';
```

| Grant | Purpose |
|---|---|
| `GRANT USAGE ON SCHEMA public TO app_user;` | Allows use of the public schema |
| `GRANT SELECT, INSERT, UPDATE, DELETE ON TABLE public.employees TO app_user;` | Allows CRUD on employees (table only) |
| `GRANT SELECT ON TABLE public.departments TO app_user;` | Allows read-only access to departments |
| `GRANT USAGE, SELECT ON SEQUENCE public.employee_id_seq TO app_user;` | Allows use of the sequence for IDs |
| `GRANT EXECUTE ON FUNCTION public.calculate_bonus() TO app_user;` | Allows executing a specific function |

### Result

The app_user role can **only** perform what it is explicitly granted.
It cannot alter, drop, or access objects it does not have permission to use.

## Least Privilege in Action

### Least Privilege (Good)

**app_user**
(Application Role)

- **USAGE** on schema
- **SELECT, INSERT, UPDATE, DELETE** on needed tables
- **USAGE** on required sequences
- **EXECUTE** on specific functions

**Access Only to Required Objects and Actions**

Reduced risk and impact

### Over-Privileged (Risky)

**app_user**
(Over-Privileged)

- **ALL PRIVILEGES** on schema
- **ALL PRIVILEGES** on all tables
- **ALL PRIVILEGES** on sequences
- **CREATE, DROP, ALTER, ...**

**Access Beyond Requirements**

Higher risk and potential damage

### Key Takeaway

Grant the **minimum permissions** needed for the job.
Review permissions regularly and remove what is no longer required.

## Best Practices

- **Use Roles, Not Users**
  Assign permissions to roles and grant roles to users.
- **Apply Least Privilege**
  Grant only what is necessary to perform the task.
- **Review Regularly**
  Audit permissions and remove unnecessary access.
- **Monitor and Log**
  Enable logging to track access and changes.
- **Separate Duties**
  **Use different roles for development, reporting, and administration.**

# TRANSACTIONS OVERVIEW

*A transaction groups one or more statements into a single all-or-nothing unit of work.*

```
BEGIN;
UPDATE account SET balance = balance - 100 WHERE account_id = 1;
UPDATE account SET balance = balance + 100 WHERE account_id = 2;
COMMIT;  -- or ROLLBACK; to undo both statements
```

KEY TAKEAWAY  BEGIN...COMMIT groups statements so they succeed or fail together -- essential the moment more than one row must change consistently, like a transfer between two accounts.

# Transactions Overview

Transactions ensure **data integrity** by treating a series of operations as a single unit of work.

## What is a Transaction?

A transaction is a sequence of one or more SQL operations executed as a single logical unit of work.
Either all operations succeed, or none are applied.

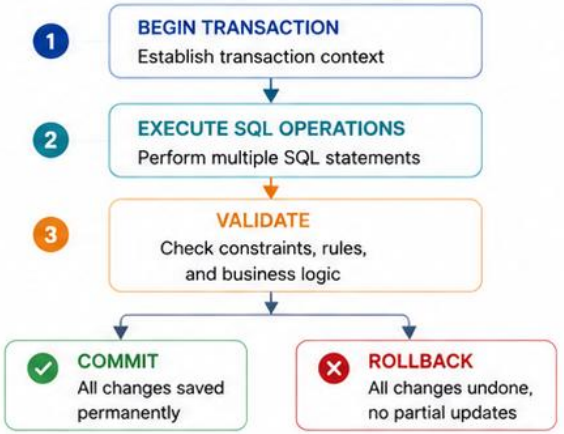
## Key Properties

- Maintains data consistency and reliability
- Provides isolation between concurrent operations
- Ensures database remains in a valid state
- Controlled with **COMMIT**, **ROLLBACK**, and **SAVEPOINT**

## Transaction Lifecycle


If any operation fails, use **ROLLBACK** to prevent partial updates and keep the database consistent.

## How a Transaction Works




## ACID Properties

**A — Atomicity**
All operations in the transaction succeed or none do.

**C — Consistency**
Database remains in a valid state before and after the transaction.

**I — Isolation**
Concurrent transactions do not interfere with each other.

**D — Durability**
Once committed, changes are permanent, even after failures.

## Example (PostgreSQL)

**Successful Transaction**

```
BEGIN;
UPDATE accounts
   SET balance = balance - 500
 WHERE account_id = 1001;

UPDATE accounts
   SET balance = balance + 500
 WHERE account_id = 1002;

COMMIT;  -- Changes are saved
```

**Failed Transaction (Rolled Back)**

```
BEGIN;
UPDATE accounts
   SET balance = balance - 500
 WHERE account_id = 1001;

UPDATE accounts
   SET balance = balance + 500
 WHERE account_id = 9999;   -- Invalid account

ROLLBACK;   -- No changes applied
```

## Common Use Cases

- Money transfers between accounts
- Order processing
- Inventory updates
- Bulk data operations
- Any operation requiring data consistency

## Best Practices

**Keep transactions short**
Minimize lock contention and improve performance.

**Do not mix user interaction within transactions**
Collect input first, then start the transaction.

**Handle exceptions properly**
Rollback on error to maintain consistency.

**Use appropriate isolation level**
Balance consistency needs with performance.

**Monitor long-running transactions**
They can block other operations.

# ACID IN POSTGRESQL

*ACID is the four-part guarantee PostgreSQL makes about every transaction: Atomicity, Consistency, Isolation, Durability.*

### Atomicity

All of a transaction's statements succeed together, or none of them do -- no partial results.

### Consistency

A transaction can only move the database from one valid state to another -- constraints are never violated.

### Isolation

Concurrent transactions don't see each other's uncommitted changes -- enforced by MVCC.

### Durability

Once COMMIT returns, the change survives a crash -- it's written to the WAL before confirming.

KEY TAKEAWAY  PostgreSQL's ACID guarantees are what let a developer trust that a committed transaction really happened, and a rolled-back one really didn't -- even after a crash.

# ACID in PostgreSQL

PostgreSQL ensures reliable transactions using **ACID** properties.

## What is ACID?

ACID is a set of properties that guarantee reliable processing of database transactions, even in the event of errors, failures, or concurrent access.

**PostgreSQL is fully ACID-compliant.**

It uses MVCC, Write-Ahead Logging (WAL), and robust locking mechanisms to protect your data.

## Why ACID Matters

- Prevents data corruption
- Maintains consistency under concurrent load
- Supports safe recovery from failures
- Builds trust in your data and applications

## The ACID Properties

**A — Atomicity**
All operations in a transaction are treated as a single unit. Either all succeed or none do.

**C — Consistency**
A transaction brings the database from one valid state to another, preserving all rules and constraints.

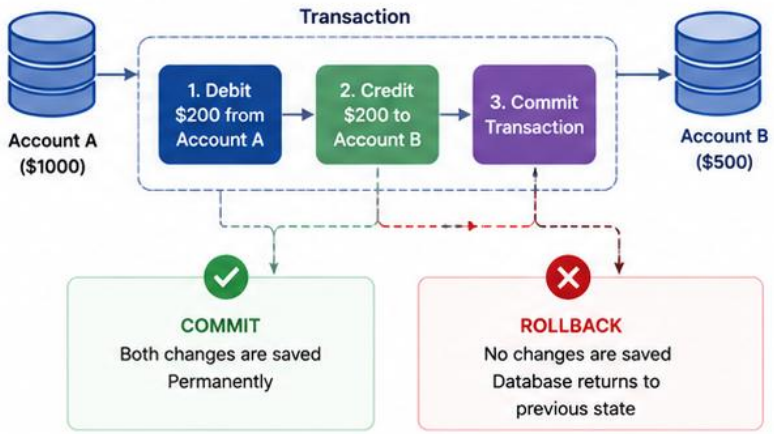
**I — Isolation**
Concurrent transactions do not interfere with each other. Each transaction appears to run in isolation.

**D — Durability**
Once a transaction is committed, changes are permanent—even if the system crashes.

### How PostgreSQL Achieves ACID

- MVCC (Multi-Version Concurrency Control) for isolation
- WAL (Write-Ahead Logging) for durability
- Locks and row-level locking for consistency and isolation
- Crash recovery and checkpoints for atomicity and durability

## Example: Bank Transfer Transaction




## Example in PostgreSQL (SQL)

```
BEGIN;
UPDATE accounts
  SET balance = balance - 200
  WHERE id = 1;  -- Debit Account A
UPDATE accounts
  SET balance = balance + 200
  WHERE id = 2;  -- Credit Account B

-- If everything is correct
COMMIT;   -- Changes are saved

-- If any error occurs
ROLLBACK;   -- No changes applied
```

## Best Practices

**Keep Transactions Short**
Long transactions increase lock contention and impact performance.

**Do Not Mix User Interaction**
Collect input first, then start the transaction.

**Handle Exceptions**
Use ROLLBACK on errors to maintain consistency.

**Test Failures**
Regularly test rollback and recovery scenarios.

**Monitor Performance**
Track long-running transactions and lock waits.

# ENTERPRISE SCHEMA DESIGN BEST PRACTICES

*Six practices that separate a schema that merely works from one that survives production.*

| Practice | In Short |
|---|---|
| **Business Requirements** | Model real entities and rules -- design for today and for growth. |
| **Model Data Correctly** | Right types, normalized to 3NF, surrogate keys. |
| **Data Integrity** | PK, FK, NOT NULL, UNIQUE, CHECK enforced at the database level. |
| **Index Smartly** | Index foreign keys and hot filter columns -- avoid over-indexing. |
| **Scalability & Security** | Plan for growth; enforce least-privilege roles. |
| **Maintainability & Governance** | Consistent naming, documented schema, clear ownership. |

KEY TAKEAWAY  Every enterprise schema decision in this course traces back to one of these six practices -- treat this table as a checklist to review before any production migration.

# Enterprise Schema Design Best Practices

Build scalable, maintainable, secure, and high-performance PostgreSQL schemas.

## Guiding Principles

**Model the Business**
Design schemas that reflect business entities, rules, and processes.

**Plan for Scale**
Design for growth in data volume, users, and performance.

**Ensure Data Integrity**
Use constraints and relationships to enforce correctness.

**Keep it Simple**
Avoid unnecessary complexity and premature optimization.

**Secure by Design**
Apply least privilege and protect sensitive data.

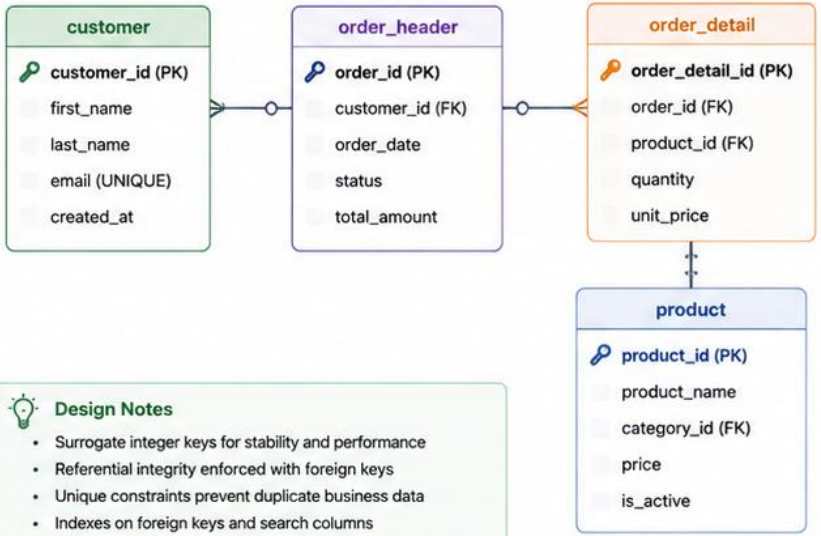
## Good Schema Design Benefits

- Consistent and accurate data
- High performance and scalability
- Easier maintenance and evolution
- Stronger security and compliance
- Lower total cost of ownership

## Best Practices Checklist

- **Use meaningful**, consistent naming for schemas, tables, and columns.
- **Follow normalization** (3NF) to reduce redundancy and anomalies.
- **Define primary keys** for all tables using surrogate keys when appropriate.
- Use **foreign keys** to enforce relationships and maintain referential integrity.
- Apply appropriate **data types** and sizes—avoid oversized columns.
- Enforce NOT NULL, UNIQUE, CHECK, and DEFAULT constraints.
- Create **indexes** strategically on joins, filters, and unique columns.
- **Avoid over-indexing**—review and remove unused indexes.
- Design for **auditability** (created_by, created_at, updated_at, etc.).
- Plan for **data lifecycle**—archiving, purging, and retention policies.
- **Use schemas** to organize domains and control access.
- **Document** the data model, rules, and relationships.

## Example: Well-Designed Schema




### Design Notes

- Surrogate integer keys for stability and performance
- Referential integrity enforced with foreign keys
- Unique constraints prevent duplicate business data
- Indexes on foreign keys and search columns
- Clear separation of customer, order, and product domains

## Common Pitfalls to Avoid

- Using reserved words or unclear names
- Storing multiple values in a single column
- Skipping constraints or primary keys
- Over-normalizing creating unnecessary complexity
- Missing or poorly designed indexes
- Ignoring growth, archiving, and retention needs

## Additional Recommendations

- Review and refactor schemas regularly to adapt to changing business needs.
- Use naming standards (e.g., snake_case) and consistent conventions across the organization.
- Add comments to tables and columns to explain purpose and business meaning.
- Monitor query performance and adjust schema, indexes, and statistics as needed.
- Leverage tools like pgAdmin, ER diagrams, and migration frameworks (Flyway/Liquibase).
- Collaborate with stakeholders, DBAs, and developers during design and reviews.

# COMMON DATABASE DESIGN MISTAKES

*The same handful of mistakes account for most schema problems found in real code review.*

| Mistake | Why It Hurts | Fix |
|---|---|---|
| **Storing money as REAL/FLOAT** | Rounding error corrupts balances over time. | Use NUMERIC(19,2). |
| **No foreign keys ('app enforces it')** | Orphan rows appear the moment any code path has a bug. | Declare real FOREIGN KEY constraints. |
| **Everything nullable by default** | Hides which data is truly required. | Add NOT NULL wherever a business rule requires it. |
| **Indexing every column** | Slows every write; wastes storage. | Index with intent, based on real query patterns. |
| **No naming convention** | Slows every future change and review. | Consistent snake_case, documented schema. |

KEY TAKEAWAY  Each row in this table is a real mistake this module already showed how to avoid -- treat it as a pre-submission checklist for Lab 37.

# TRY IT YOURSELF — EXERCISE 6: LAB 37 READINESS SELF-CHECK

*Pre-lab gate: sweep your schema against the common-mistakes slide, plan the negative tests, and confirm the five earlier notes files are complete.*

## STEPS (notes/lab37-readiness.md)

| Step | What To Do |
|---|---|
| **1 — Mistake Sweep** | Check your schema against the common-mistakes slide and fix what matches. |
| **2 — Negative Tests** | Name three inserts that must fail, and the constraint that rejects each. |
| **3 — Rebuild Proof** | Say how you will prove the schema can be dropped and recreated cleanly. |
| **4 — Pass Mark** | Mark Pass only if all five earlier notes files are complete, not stubbed. |

## Negative tests

```
duplicate email      -> unique violation
status 'PENDING'     -> check violation
account with unknown customer_id -> foreign key violation
```

TRY IT NOW — Complete Exercise 6 before Lab 37: exercises/exercise-06-lab37-readiness.md (workspace: examples/module-37-exercises).

# Common Database Design Mistakes

Avoid these pitfalls to build **scalable, reliable, and high-performance** PostgreSQL databases.

| Mistake | Why It's a Problem | Better Approach |
|---|---|---|
| **1. Poor or Missing Primary Keys**<br>Tables without primary keys or with non-unique keys. | Leads to duplicate data, update anomalies, poor performance, and replication issues. | Always define a primary key using surrogate keys (e.g., BIGSERIAL/UUID) when appropriate. |
| **2. Over-Normalization**<br>Splitting data into too many tables without business need. | Increases complexity, more joins, and slower queries. | Normalize to 3NF, but denormalize strategically for performance when justified. |
| **3. Using SELECT * Everywhere**<br>Retrieving unnecessary columns in queries. | Consumes more I/O and network bandwidth. Impacts query performance. | Select only the required columns. Be explicit in queries and APIs. |
| **4. Incorrect Data Types**<br>Using generic or oversized data types (e.g., TEXT for IDs). | Wastes storage, slows down queries, and reduces data integrity. | Use appropriate data types (e.g., INTEGER, TIMESTAMP, NUMERIC, VARCHAR(n)). |
| **5. Missing or Poorly Designed Indexes**<br>No indexes on key columns or wrong index strategy. | Causes full table scans and poor performance. | Index foreign keys and frequently filtered, sorted, or joined columns. Avoid over-indexing. |
| **6. NULL Overuse**<br>Allowing NULLs when not necessary. | Complicates logic, causes unexpected results, and wastes space. | Use NOT NULL with appropriate DEFAULT values where possible. |
| **7. Weak Constraints**<br>Missing constraints (FK, UNIQUE, CHECK) on tables. | Allows invalid data and breaks referential integrity. | Define FOREIGN KEY, UNIQUE, CHECK, and DEFAULT constraints consistently. |
| **8. Ignoring Query Performance Early**<br>Designing schema without considering query patterns. | Leads to redesign, high query latency, and scalability issues. | Understand query patterns early. Design schema, indexes, and partitions accordingly. |
| **9. No Archiving or Partitioning Plan**<br>Storing all data in a single table indefinitely. | Leads to huge tables, slow queries, and maintenance challenges. | Plan for partitioning, archiving, and retention policies from the start. |
| **10. Mixing Business Rules in the Database**<br>Embedding complex business logic in schema (triggers/procedures). | Hard to maintain, test, and evolve. Tight coupling. | Keep business logic in the application layer. Use the database for data integrity only. |

## The Impact of Poor Design

- Poor Performance and Scalability
- Data Inconsistency and Integrity Issues
- Higher Development and Maintenance Cost
- Difficult Reporting and Analytics
- Harder to Evolve and Extend

## Good Design Outcomes

- High Performance and Scalability
- Accurate and Consistent Data
- Lower Development and Maintenance Cost
- Easier Reporting and Analytics
- Future-Proof and Flexible Schema

## Key Takeaway

Good database design is the foundation of a reliable and high-performing application. Plan well, validate early, and review continuously.

## Best Practices Reminder

- Understand your business and data before designing.
- Keep it simple, consistent, and well-documented.
- Review and test queries during design.
- Monitor performance and adjust indexes and schema as needed.
- Collaborate with DBAs, architects, and developers during design reviews.

# LAB OVERVIEW — POSTGRESQL DATABASE FUNDAMENTALS

*Create CRM DB/schema/tables with PKs, FKs, constraints, indexes, seeds |*

## BUSINESS SCENARIO

- Angular (Labs 33–36) talks to Spring REST; Spring will persist to PostgreSQL. Leadership freezes:

## LEARNING OBJECTIVES

- Create a PostgreSQL database/schema and CRM tables with named constraints Model one-to-many Customer→Account with FK indexes Seed deterministic fixtures (`CUS-1001`, `CUS-1002`) and prove negatives Connect with `psql`/pgAdmin and document a Java JDBC URL Apply least-privilege roles and state ACID properties for CRM writes

## WHAT YOU BUILD

- `examples/lab37-crm/` — SQL scripts, compose, design notes | Named constraints · CUS-1001/CUS-1002 seeds · one negative check · notes |

**KEY TAKEAWAY Design and implement the **PostgreSQL** CRM schema before ORM: databases vs schemas, tables, PKs/FKs, UNIQUE/NOT NULL/CHECK, indexes, roles/least privilege, ACID transactions, `psql`/pgAdmin, and the JDBC URL shape Spring Boot will use in Lab 39.**

# Lab Overview – PostgreSQL Database Design

Design a **normalized**, **secure**, and **performant** schema in PostgreSQL for a real-world business scenario.

## Business Scenario

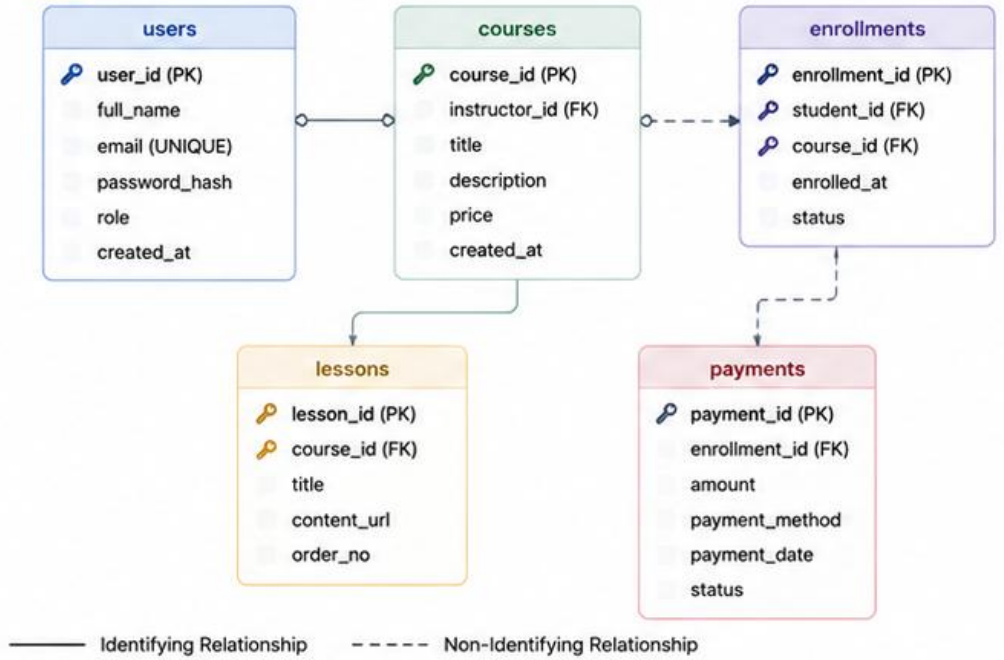
You are designing the database for an **Online Learning Platform** where students can enroll in courses, instructors create content, and payments are processed.

- Students can register and enroll in courses
- Instructors create courses and lessons
- Students make payments for paid courses
- Admins manage users and content

## Lab Objectives

- Design a normalized schema (3NF)
- Define tables with appropriate data types
- Implement primary keys, foreign keys, and constraints
- Create indexes for performance
- Apply best practices for security and maintainability
- Populate the database with sample data
- Write queries to validate design and performance

## High-Level Schema




## Lab Environment

**Database**
PostgreSQL 15+

**Tools**
pgAdmin / psql

**Provided**
Sample data sets, SQL scripts, and requirements document

**Deliverables**
SQL script, ER diagram, queries, and report

## What You Will Practice

- Creating schemas and tables
- Applying constraints and relationships
- Choosing proper data types
- Indexing strategy
- Normalizing data
- Writing queries and validating performance

## Lab Tasks

1. **Requirements Review** — Understand business requirements and identify entities.
2. **Conceptual Design** — Create ER diagram and identify relationships and cardinalities.
3. **Logical Design** — Convert ERD to relational schema and normalize to 3NF.
4. **Physical Design** — Define data types, constraints, indexes, and sequences.
5. **Implementation** — Create schema in PostgreSQL using SQL scripts.
6. **Validation & Testing** — Insert sample data, write queries, and analyze performance.
7. **Documentation** — Prepare ERD, data dictionary, and design rationale.

## Key Learning Outcomes

- Design normalized schemas that reduce redundancy.
- Implement constraints that ensure data integrity.
- Create indexes that improve query performance.
- Apply security best practices with roles and least privilege.
- Evaluate and optimize queries using EXPLAIN ANALYZE.

# LAB TASKS AND DELIVERABLES

*PostgreSQL Database Fundamentals — implementation steps, deliverables, and the evaluation rubric.*

| # | Implementation Step |
|---|---|
| **1** | Open starter/README.md. |
| **2** | Copy starter/ into java-bootcamp/examples/lab37-crm. |
| **3** | Fill every -- TODO in database/*.sql. |
| **4** | Smoke: compose up → apply scripts → seed SELECT → one negative check. |
| **5** | Evidence under notes/screenshots/lab-37/. |
| **6** | ER notes with cardinalities (database/er-diagram.md) |
| **7** | PostgreSQL Docker (or shared) runtime |
| **8** | Least-privilege crm_app role script |
| **9** | DDL: CUSTOMER + ACCOUNT (+ indexes); full path: ADDRESS + HISTORY |
| **10** | Seed Amina CUS-1001 ACTIVE + account; Ravi CUS-1002 PROSPECT |
| **11** | Negative verify (duplicate email / bad status / orphan FK) |
| **12** | Drop/recreate proof + design-decisions.md |

## EXPECTED DELIVERABLES

- ER notes with cardinalities (database/er-diagram.md) PostgreSQL Docker (or shared) runtime Least-privilege crm_app role script DDL: CUSTOMER + ACCOUNT (+ indexes); full path: ADDRESS + HISTORY Seed Amina CUS-1001 ACTIVE + account; Ravi CUS-1002 PROSPECT Negative verify (duplicate email / bad status / orphan FK) Drop/recreate proof + design-decisions.md JDBC URL note for Java; no passwords in Git

## RUBRIC (100 MARKS)

- Environment 10 · Core Impl 30 · Integration/Config 15 · Failure Handling 15 · Verification 10 · Security/Prod Awareness 10 · Docs/Evidence 10

**KEY TAKEAWAY Fixtures CUS-1001 (Amina Khan, ACTIVE) and CUS-1002 (Ravi Singh, PROSPECT) anchor Lab 37; complete pre-lab exercises 1→6 first, then the graded lab.**

# Lab Tasks and Deliverables

Complete the tasks to **design, implement, and validate** a robust PostgreSQL database schema.

## Lab Tasks

1. **Requirements Review**
   Review the business scenario and identify key entities, attributes, and rules.
2. **Conceptual Design (ERD)**
   Create an ER diagram showing entities, relationships, and cardinalities.
3. **Logical Design**
   Map the ERD to a relational schema and normalize to 3NF.
4. **Physical Design**
   Define tables with appropriate data types, primary keys, foreign keys, constraints, indexes, and sequences.
5. **Schema Implementation**
   Write and execute SQL scripts to create schemas, tables, constraints, indexes, and sequences.
6. **Data Population**
   Insert provided sample data and any additional lookup/reference data.
7. **Validation & Testing**
   Run queries to validate relationships, constraints, and performance.
8. **Performance Analysis**
   Analyze slow queries using EXPLAIN ANALYZE and add indexes if needed.
9. **Documentation**
   Document the schema, data dictionary, and design decisions.

## Deliverables

| Deliverable | Descriptios |
|---|---|
| ER Diagram | High-quality ER diagram (PNG/PDF) showing entities, relationships, and cardinalities. |
| SQL Scripts | Complete SQL scripts to create schemas, tables, constraints, indexes, sequences, and sample data. |
| Data Dictionary | Data dictionary listing tables, columns, data types, constraints, keys, and descriptions. |
| Sample Queries | SQL queries demonstrating CRUD operations, joins, and business rules. |
| Performance Report | Query performance analysis using EXPLAIN ANALYZE with observations and improvements. |
| Design Rationale | Document summarizing design decisions, assumptions, and trade-offs. |
| README | Instructions to set up and run the database and scripts. |

## Evaluation Focus

- **Correctness**
  Accurate model, constraints, and relationships.
- **Normalization**
  Proper normalization to 3NF.
- **Data Integrity**
  Appropriate constraints and referential integrity.
- **Performance**
  Effective indexing and query performance.
- **Documentation**
  Quality and completeness of documentation.
- **Dest Practices**
  Use of naming standards, data types, and design principles.

## Success Criteria

- Schema is normalized to **3NF** with minimal redundancy.
- All constraints and relationships work as expected.
- Sample data inserts without errors.
- Queries return correct results and perform efficiently.
- Documentation is clear, complete, and well-structured.

## Helpful Tips

- Start with a clear understanding of the business rules.
- Draw the ERD before writing any SQL.
- Choose the right data types for accuracy and performance.
- Define keys and constraints early.
- Use meaningful names for schemas, tables, and columns.
- Validate with real queries and edge cases.

## Recommended Workflow

1. **1. Understand**
   Read requirements and identify entities.
2. **2. Design**
   Create ERD and logical schema.
3. **3. Build**
   Write SQL scripts and implement schema.
4. **4. Populate**
   Insert sample and reference data.
5. **5. Validate**
   Run queries and verify results.
6. **6. Optimize**
   Analyze performance and tune schema.
7. **7. Document**
   Prepare and submit all deliverables.

# MODULE 37 SUMMARY

*You can now design, constrain, and connect to a normalized PostgreSQL schema.*

## KEY CONCEPTS COVERED

### Fundamentals & Architecture

Relational concepts and how PostgreSQL organizes databases, schemas, and processes.

### Data Types

Numeric, character, boolean, date/time, UUID, and JSON/JSONB types.

### Keys & Constraints

Primary keys, foreign keys, UNIQUE, NOT NULL, CHECK, and DEFAULT values.

### Relationships & Normalization

One-to-one, one-to-many, many-to-many, and reducing redundancy.

### Indexes & Sequences

B-Tree, unique, and composite indexes; sequences and identity columns.

### Connectivity & Governance

psql, pgAdmin, JDBC, users, roles, transactions, and ACID.

## MODULE FLOW RECAP

1. Model
2. Constrain
3. Relate
4. Index
5. Connect

KEY TAKEAWAY  You can now design a normalized PostgreSQL schema, enforce its rules with constraints, index it for performance, and connect to it from psql and Java.

# MODULE SUMMARY

## Module 13: REST API Design with Java

In this module, you explored the principles and best practices for designing high-quality, scalable, and consistent RESTful APIs using Java and HTTP. You learned how to model resources, design clean URIs, choose the right HTTP methods, structure requests and responses, handle errors, and document APIs using OpenAPI.

### WHAT YOU LEARNED

- REST Principles and Constraints
- Client–Server and Stateless Communication
- Resource-Based API Design
- HTTP Methods and Status Codes
- Request & Response JSON and Headers
- Pagination, Filtering, and Sorting
- Error Handling and API Error Design
- API Versioning Strategies
- OpenAPI and API-First Development
- Best Practices and Common Mistakes

### KEY TAKEAWAYS

- REST APIs enable scalable, interoperable, and maintainable enterprise integrations.
- A well-designed API starts with clear resources, clean URIs, and correct use of HTTP methods.
- Consistent request/response structures and meaningful status codes improve reliability and developer experience.
- Pagination, filtering, sorting, and versioning are essential for real-world APIs.
- OpenAPI documentation ensures clarity, standardization, and collaboration.
- Following best practices and avoiding common mistakes leads to robust and sustainable APIs.

**You are now equipped with the foundational knowledge to design RESTful APIs that are consistent, scalable, secure, and aligned with enterprise standards.**

# KNOWLEDGE CHECK (1 OF 2)

*Test your understanding of relational fundamentals, data types, and keys.*

**1. Which PostgreSQL object groups related tables under a shared namespace inside one database?**

A. A tablespace

**B. A schema**

C. A sequence

D. A role

**2. Which PostgreSQL data type should store exact monetary amounts?**

A. REAL

B. FLOAT

**C. NUMERIC(p,s)**

D. TEXT

**3. What must be true of every primary key value in a table?**

A. It may repeat across rows

B. It may be NULL for optional rows

**C. It must be unique and NOT NULL**

D. It must be a UUID

**4. A foreign key column in a child table must:**

A. Always be NOT NULL

**B. Reference an existing primary key value in the parent table (or be NULL)**

C. Be indexed automatically by PostgreSQL

D. Match the child table's own primary key

KEY TAKEAWAY  Schemas group tables inside a database; NUMERIC stores exact money; a primary key is always unique and NOT NULL; a foreign key must match an existing parent key or be NULL.

# KNOWLEDGE CHECK (2 OF 2)

*Four more questions on relationships, indexes, transactions, and connecting to PostgreSQL.*

**1. A many-to-many relationship between ORDER and PRODUCT is correctly modeled in PostgreSQL by:**

A. Adding a product_id column directly to ORDER

**B. A junction table with foreign keys to both tables**

C. Storing a comma-separated list of product IDs

D. Using a CHECK constraint

**2. Which index type does PostgreSQL use by default for CREATE INDEX?**

A. Hash

B. GIN

**C. B-Tree**

D. BRIN

**3. What does the ACID guarantee 'Atomicity' mean in PostgreSQL?**

A. Queries run in alphabetical order

**B. A transaction's statements all succeed or all roll back together**

C. Every column value is atomic and indivisible

D. Only one user can connect at a time

**4. Which JDBC connection URL correctly targets a PostgreSQL database named crmdb on localhost?**

A. jdbc:mysql://localhost:5432/crmdb

B. postgresql://localhost/crmdb

**C. jdbc:postgresql://localhost:5432/crmdb**

D. jdbc:postgres:crmdb@localhost

KEY TAKEAWAY  Many-to-many relationships need a junction table; B-Tree is PostgreSQL's default index; Atomicity means a transaction commits or rolls back as one unit; the JDBC URL form is jdbc:postgresql://host:port/database.

# KNOWLEDGE CHECK

## Module 13: REST API Design with Java

Test your understanding of key concepts from this module.

**1** **Which REST constraint means the server does not store any client context between requests?**

- A Cacheability
- B Client–Server
- C Stateless
- D Uniform Interface

**2** **Which HTTP method should be used to create a new resource?**

- A GET
- B POST
- C PUT
- D DELETE

**3** **What does an idempotent operation ensure?**

- A It is always fast
- B It can be cached
- C Multiple identical requests have the same effect as one
- D It is always safe

**4** **Which status code indicates that the request was successful and a resource was created?**

- A 200 OK
- B 201 Created
- C 204 No Content
- D 400 Bad Request

**5** **Which component of an HTTP request is used to indicate the expected response media type?**

- A Content-Type
- B Accept
- C Authorization
- D User-Agent

**6** **Which approach puts the version number directly in the URI path?**

- A Header-Based Versioning
- B Query Parameter Versioning
- C URI Versioning
- D Media Type Versioning

v1

## CHECK YOURSELF

- Review the concepts and examples in this module.
- Think about how each concept applies to real APIs.
- Revisit any topics that were challenging.
- Aim for a strong understanding before moving to the lab.

**Tip**

Good API design leads to scalable, maintainable, and developer-friendly enterprise solutions.

**Remember:** A well-designed REST API is consistent, predictable, and easy to use.
Follow the principles, use the right HTTP methods, and design with your consumers in mind.

MODULE 37 COMPLETE

*"Model the data. Enforce the rules. Connect it all together."*

# PostgreSQL Database Fundamentals

You can now design normalized PostgreSQL schemas, enforce integrity with keys and constraints, index for performance, and connect from psql and Java.